

Retrieval-Augmented Generation

Overview

1. Why RAG (LLM limits)
2. End-to-end RAG pipeline + quality levers
3. Retrieval strategies in RAG (hybrid, rerank, query rewrite)
4. Context construction (packing, dedup, citations)
5. Evaluation + debugging playbook
6. Safety (prompt injection, ACLs)
7. Advanced: iterative/token-level retrieval + long-context tradeoffs (optional/appendix)

Start with a simple prompt based interaction

Act as a slightly dramatic but brilliant professor. Explain what 'prompting' is in ChatGPT to a tired student in exactly 6 bullet points, using one cooking analogy and one 'what not to do' example, and end with a one-sentence cheat code they can remember.

- Prompting is the art of telling ChatGPT *exactly what kind of thinking you want*—your prompt is the "assignment sheet," and vague prompts are like turning in a blank page and hoping I'll divine your thesis through academic telepathy.
- A good prompt has **three bones**: **goal** (what you want), **context** (what you're working with), and **constraints** (format, length, tone, audience, tools)—because the model can improvise, but it performs best when you set the stage.
- Think of it like cooking: "Make food" gets you *something edible-ish*; "Make a 10-minute vegetarian pasta with garlic, spinach, and lemon, no dairy, one pot" gets you a dish that actually matches your hunger and your pantry.
- You can steer quality by asking for a process: "Give me two options, then choose one and justify it," or "Ask me two clarifying questions if needed," or "Show steps, then a final answer," which is basically you handing the model a lab protocol instead of vibes.
- What not to do: don't write "Help. It's due tomorrow. Make it good." (That's a cry for help, not a prompt.) Instead: "Write a 200-word intro arguing X, for a skeptical audience, using two concrete examples, in a calm tone."
- The secret power move is iteration: treat the first output as a draft, then refine with "shorter," "more formal," "add an example," "remove fluff," "focus on point #2," because prompting is less like asking an oracle and more like directing an extremely fast, slightly literal assistant.

Cheat code: "Goal + Context + Constraints + Example = good prompt."

RAG Example with Simple Prompt

Retrieved Document

Knowledge: Arctic Ocean. Although over half of Europe's original forests disappeared through the centuries of deforestation, Europe still has over one quarter of its land area as forest, such as the broadleaf and mixed forests, taiga of Scandinavia and Russia, mixed rainforests of the Caucasus and the Cork oak forests in the western Mediterranean. During recent times, deforestation has been slowed and many trees have been planted. However, in many cases monoculture plantations of conifers have replaced the original mixed natural forest, because these grow quicker. The plantations now cover vast areas of land, but offer poorer habitats for many European

Question: As of 2015, since 1990 forests have _____ in Europe and have _____ in Africa and the Americas.

A. "increased, increased" B. "increased, decreased" C. "decreased, increased" D. "decreased, decreased"

Answer: B

Complete Prompt

Will cover how this idea applies to general tool-use & reasoning

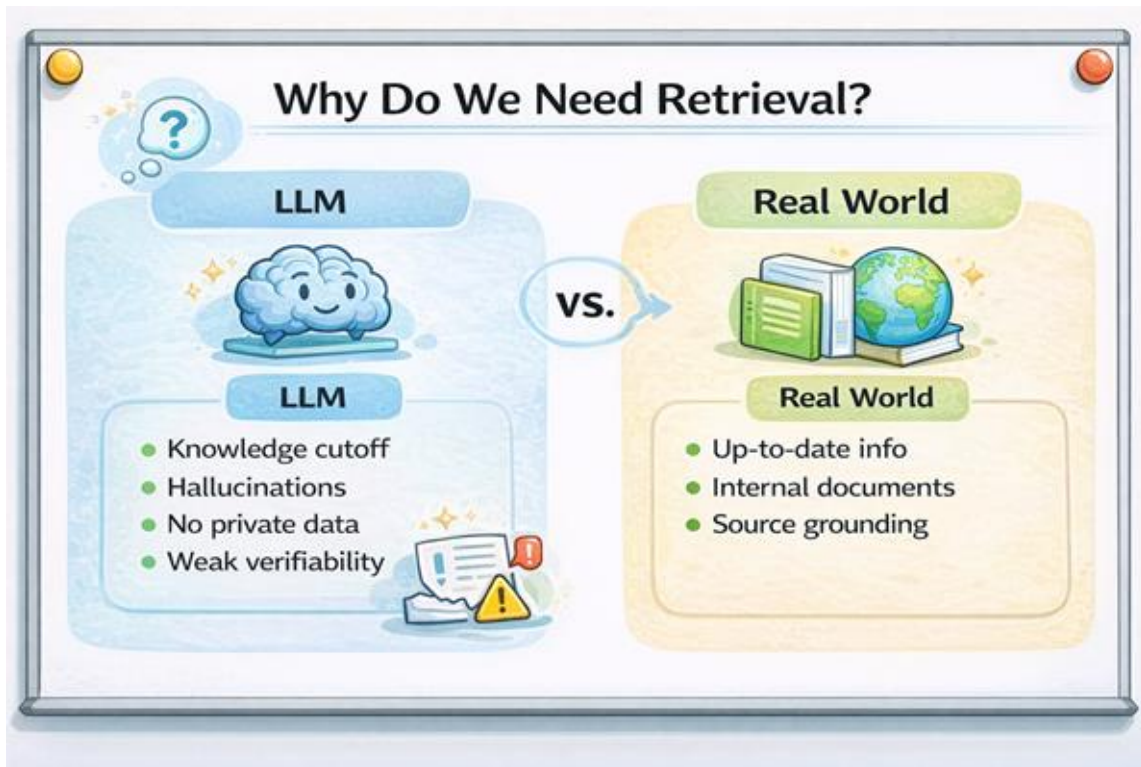
Prompts: interface between humans and LLMs

A prompt usually contains some combination of:

- Instructions (what you want the model to do)
- Context (background information)
- Data (text, code, tables, documents)
- Constraints (format, style, length, rules)

The model does not “know” what you want unless it is expressed in the prompt.

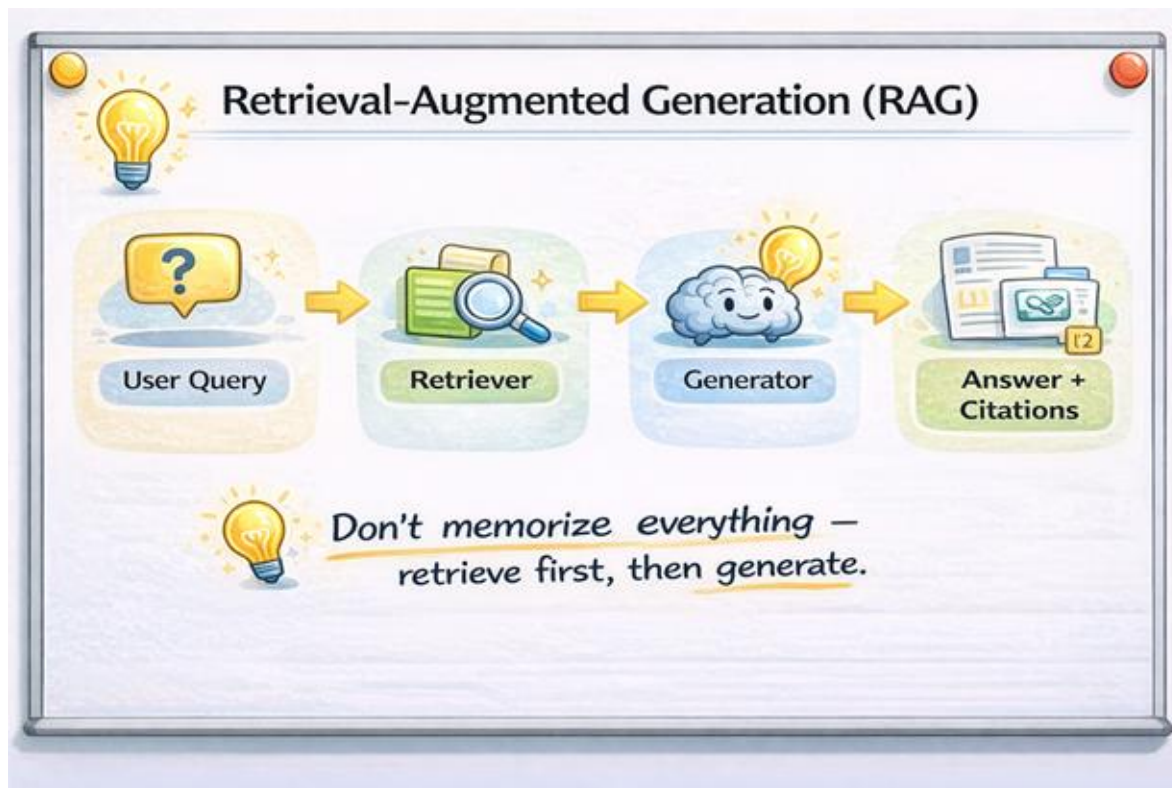
Why do we Need RAG?



Ways to get around the issues



What is RAG?



Example



You

I think Vin Diesel has been a voice actor for several characters in TV series, do you know what their names are? Please list all of them.



ChatGPT

Vin Diesel has lent his voice to several characters across various TV series and shows. Here's a list of the characters he's voiced:

1. **Groot** in "Guardians of the Galaxy Vol. 3" (2023 Movie), "The Guardians of the Galaxy Holiday Special" (2022 TV Show), "I Am Groot" (2022 Short), "Thor: Love and Thunder" (2022 Movie), "Avengers: Endgame" (2019 Movie), "Ralph Breaks the Internet: Wreck-It Ralph 2" (2018 Movie), "Avengers: Infinity War" (2018 Movie), "Guardians of the Galaxy Vol. 2" (2017 Movie), and "Guardians of the Galaxy" (2014 Movie) [\[1\]](#).
2. **Santiago** in "ARK: The Animated Series" (2023 TV Show) [\[1\]](#).
3. **Dominic Toretto** in "Fast & Furious: Spy Racers" (2019 TV Show) [\[1\]](#).

<https://www.behindthevoiceactors.com/Vin-Diesel/>

Additionally, though the website for "Big Mouth" lists Vin Diesel, it appears to be a misunderstanding or an error, as Nick Kroll is credited as the voice of Vin Diesel in that show [\[1\]](#).

<https://www.behindthevoiceactors.com/tv-shows/Big-Mouth/Vin-Diesel/>

These roles showcase Diesel's versatility and contribution to voice acting, particularly in animated series and specials, where he brings iconic characters to life with his distinctive voice.



Example

I don't like Garlic but am hungry, find me a good recipe online

Thought for 7s >

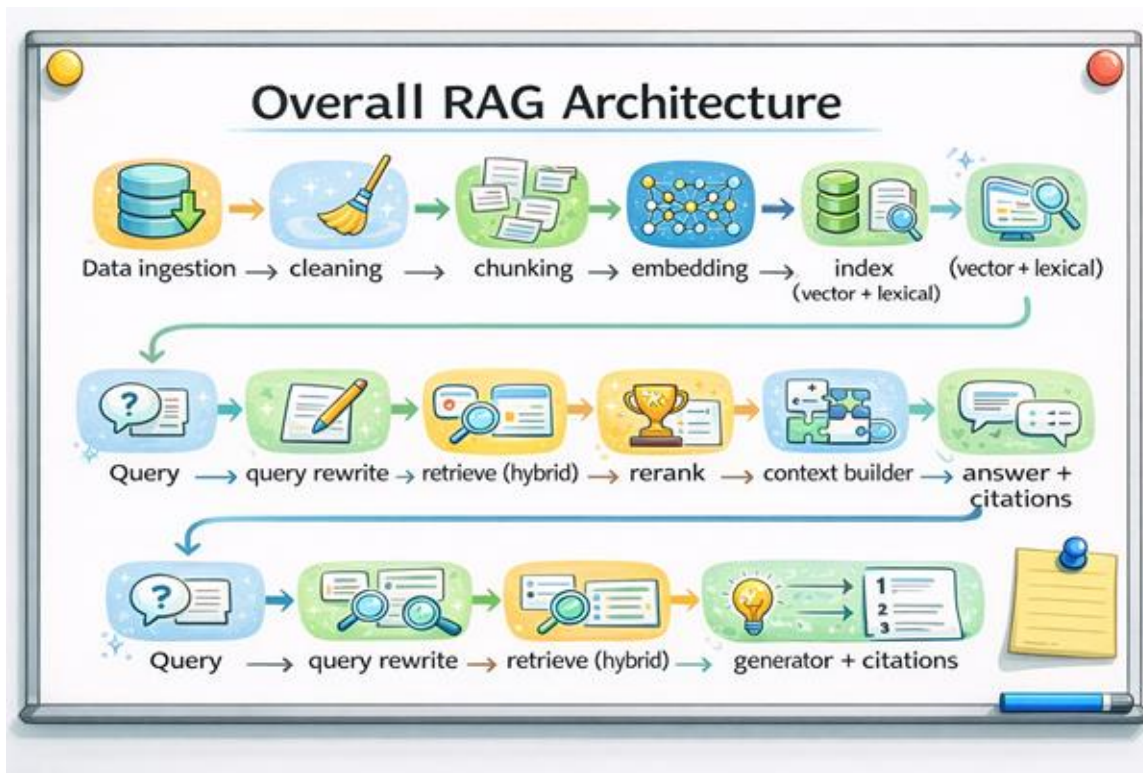
Here—garlic-free, hungry-now options you can click and cook without thinking too hard:

1. **20-minute One-Pot Lemon Spaghetti with Spinach (vegan, cozy, fast)** — bright lemon + spinach, all in one pot, very “feed me immediately” energy. [Vegan Heaven](#)
2. **Quick Lemon Spinach Pasta with a no-cook sauce (≈20 minutes)** — the sauce happens while the pasta is hot; minimal effort, maximal comfort. [Seasons and Sup...](#)
3. **Simple Lemon Herb Chicken (explicitly *without garlic*, sheet-pan-ish vibes)** — lemon + herbs + veggies + chicken; solid if you want protein and leftovers. [Ignite Nutrition](#)
4. **If you're *too tired to cook*: EatingWell's “no-recipe dinners” ideas (power bowls, one-pot pasta concepts, microwave potato + toppings)** — low brainpower required. [EatingWell](#)

If you tell me **veg vs chicken** and your **time limit (10/20/30 min)**, I'll pick the single best one and summarize the steps in a tiny “do-this-next” list.

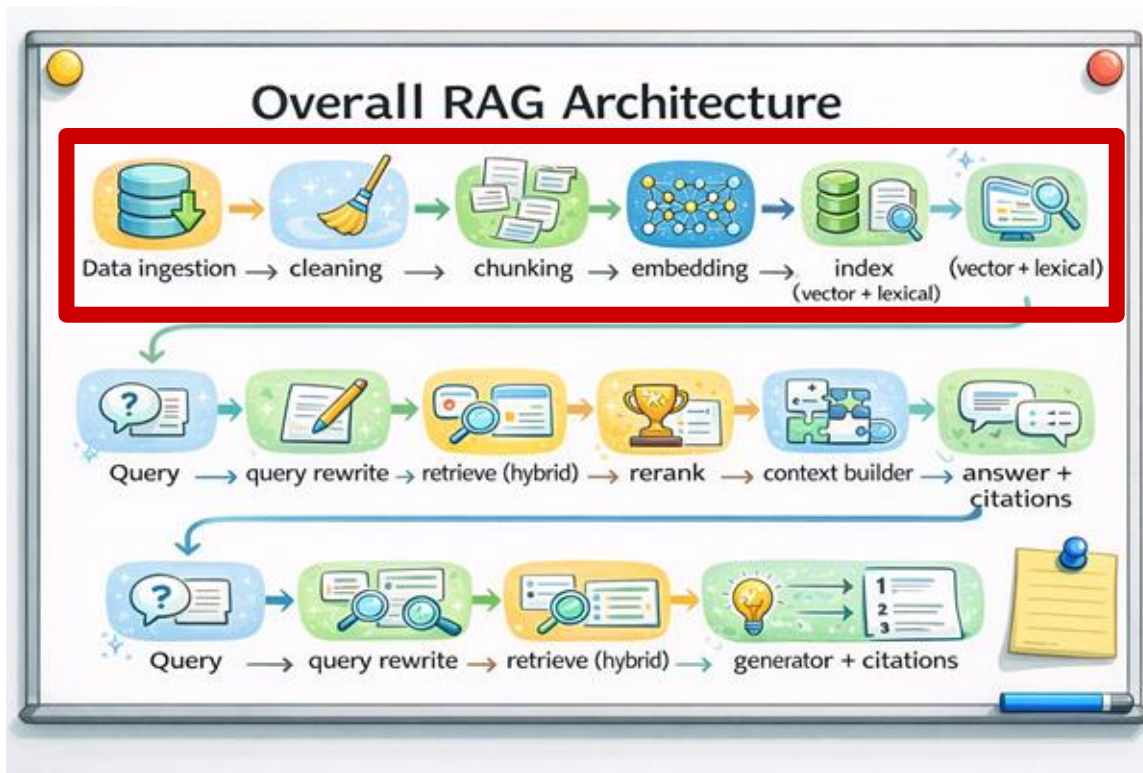
     ...  Sources 

RAG Architecture

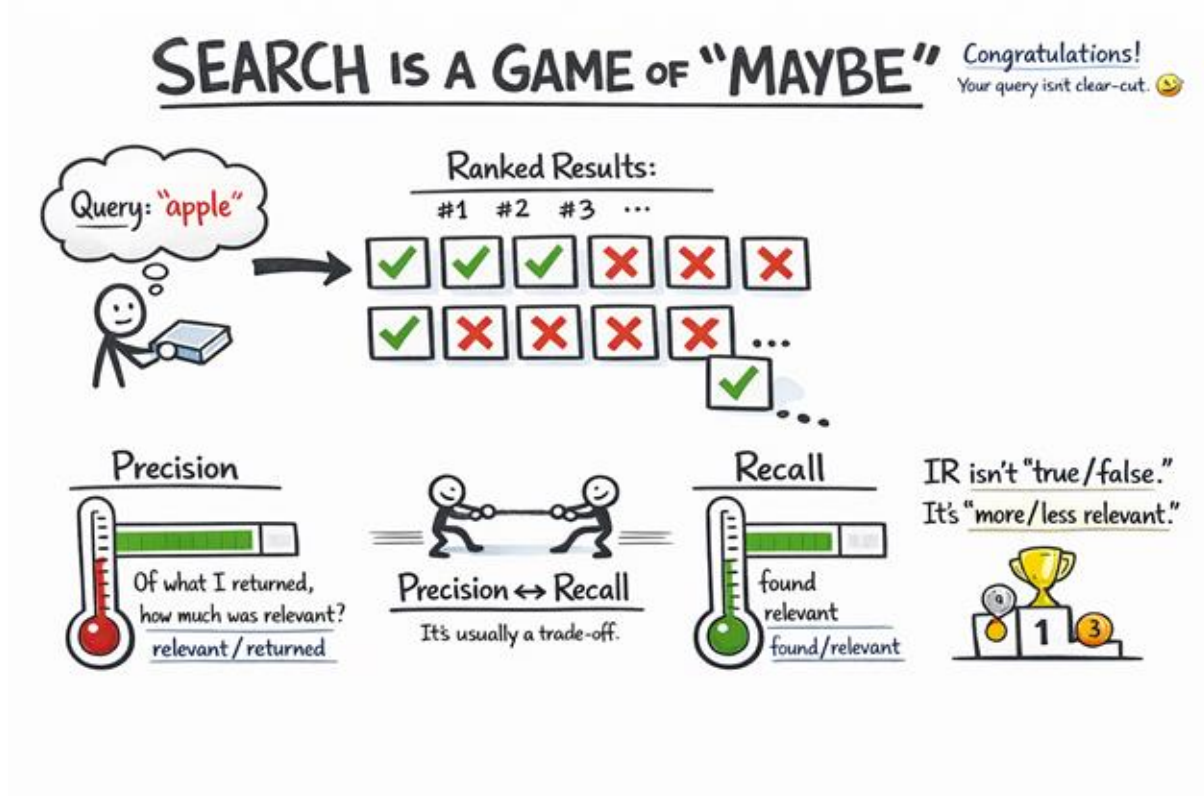


Retrieval Methods

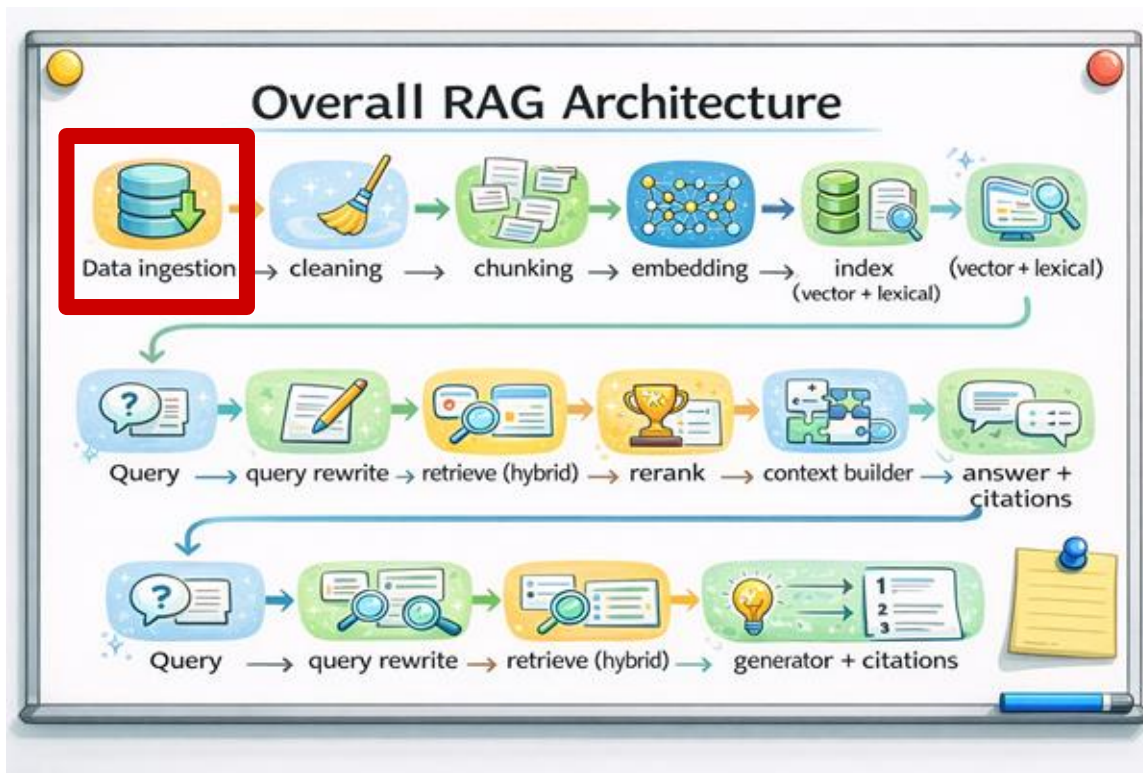
RAG Architecture



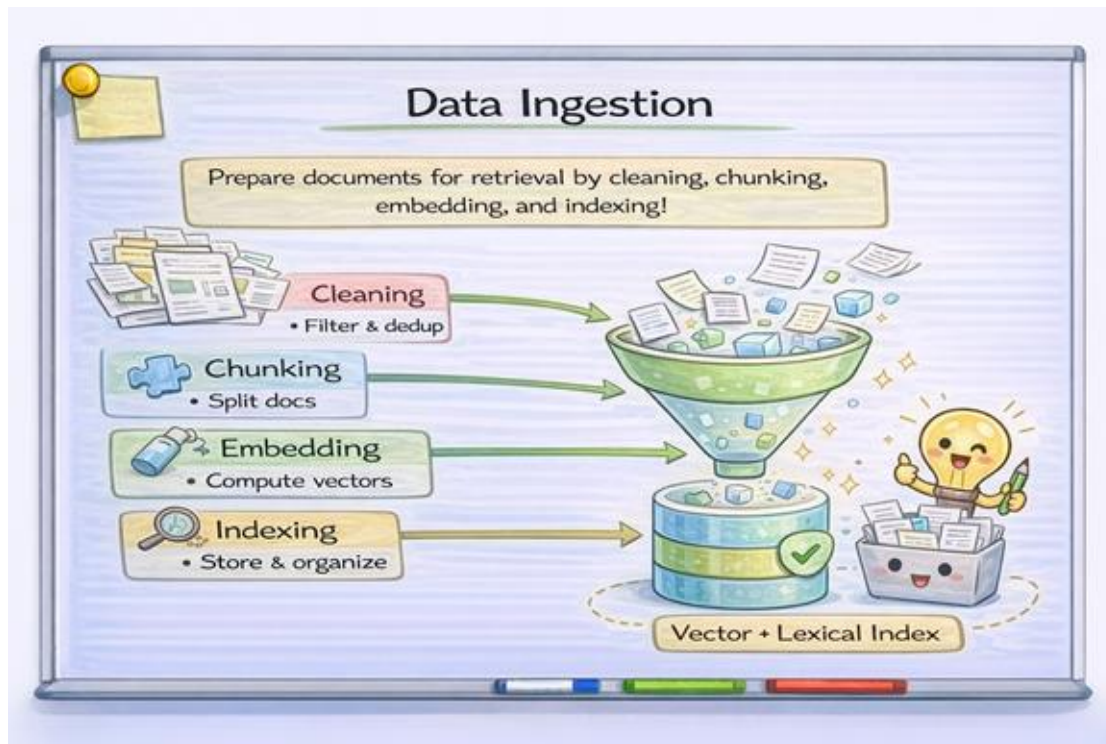
What is Information Retrieval?



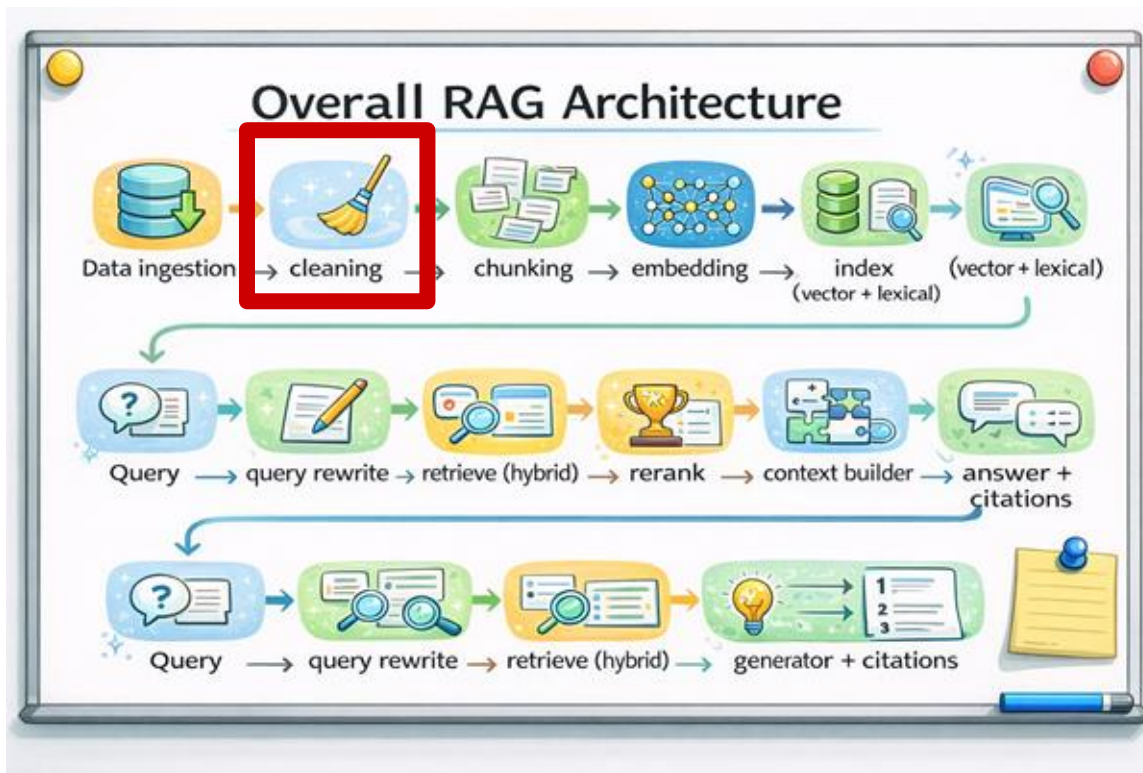
RAG Architecture



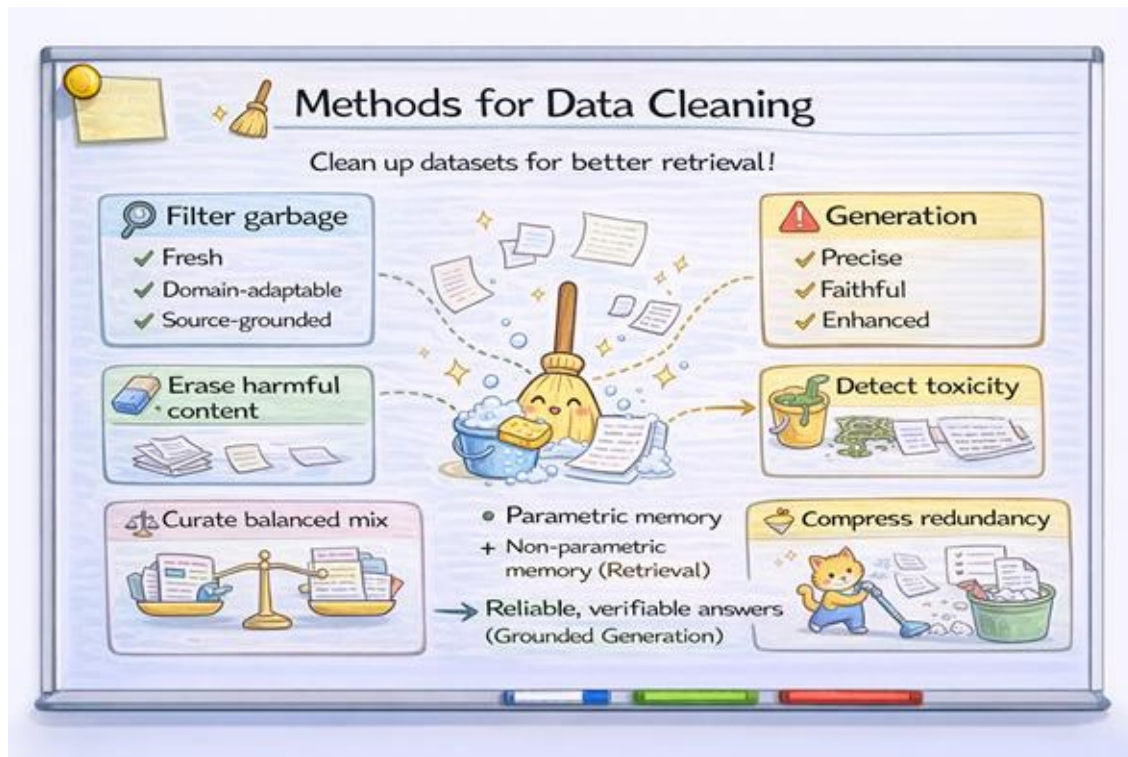
Data Ingestion



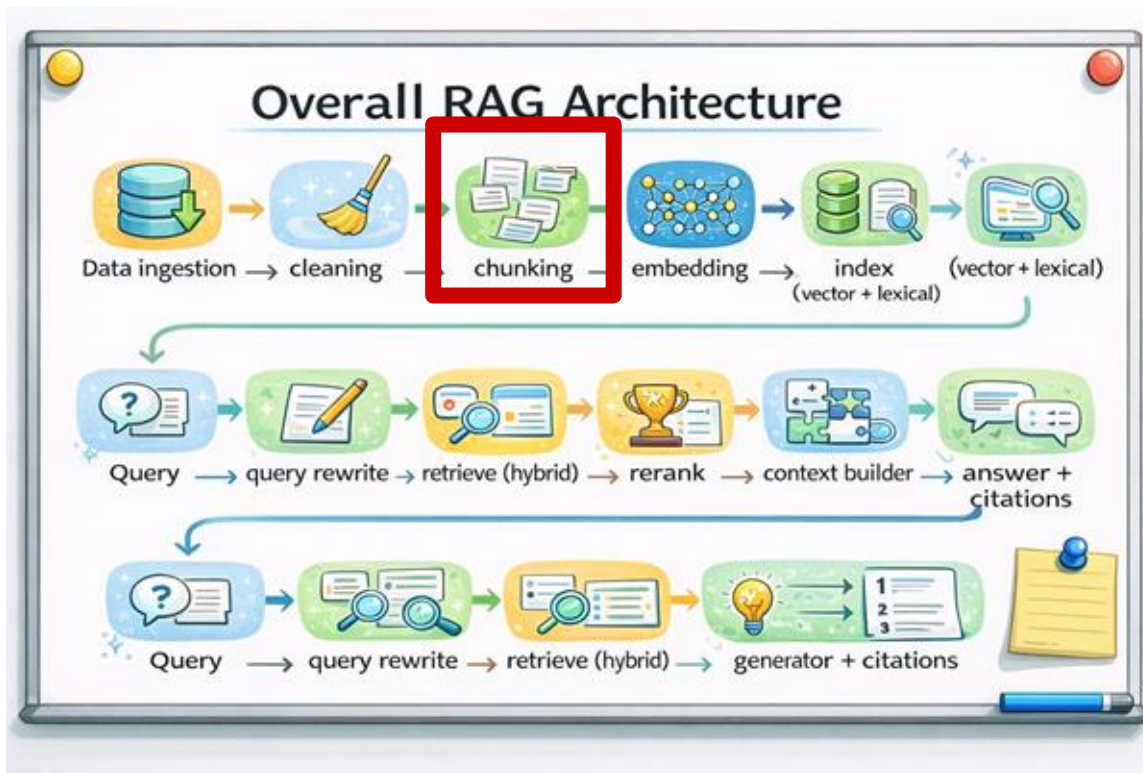
RAG Architecture



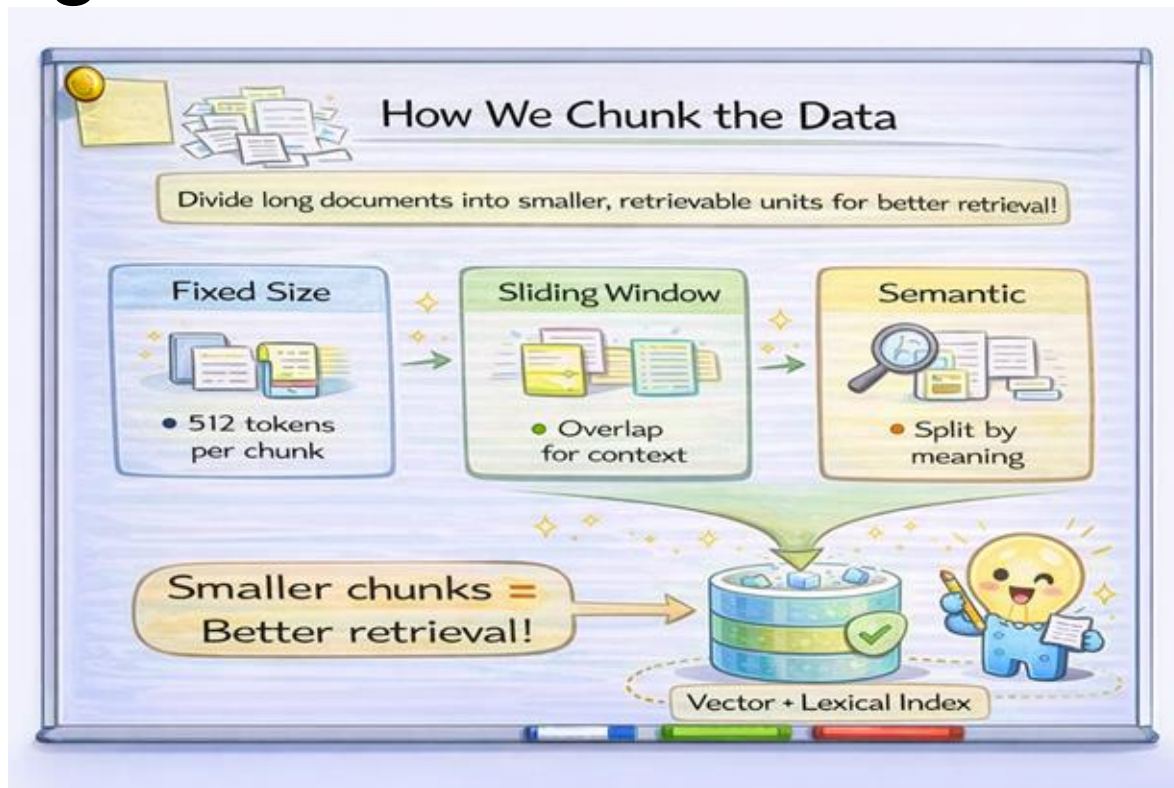
Let's do some cleaning...



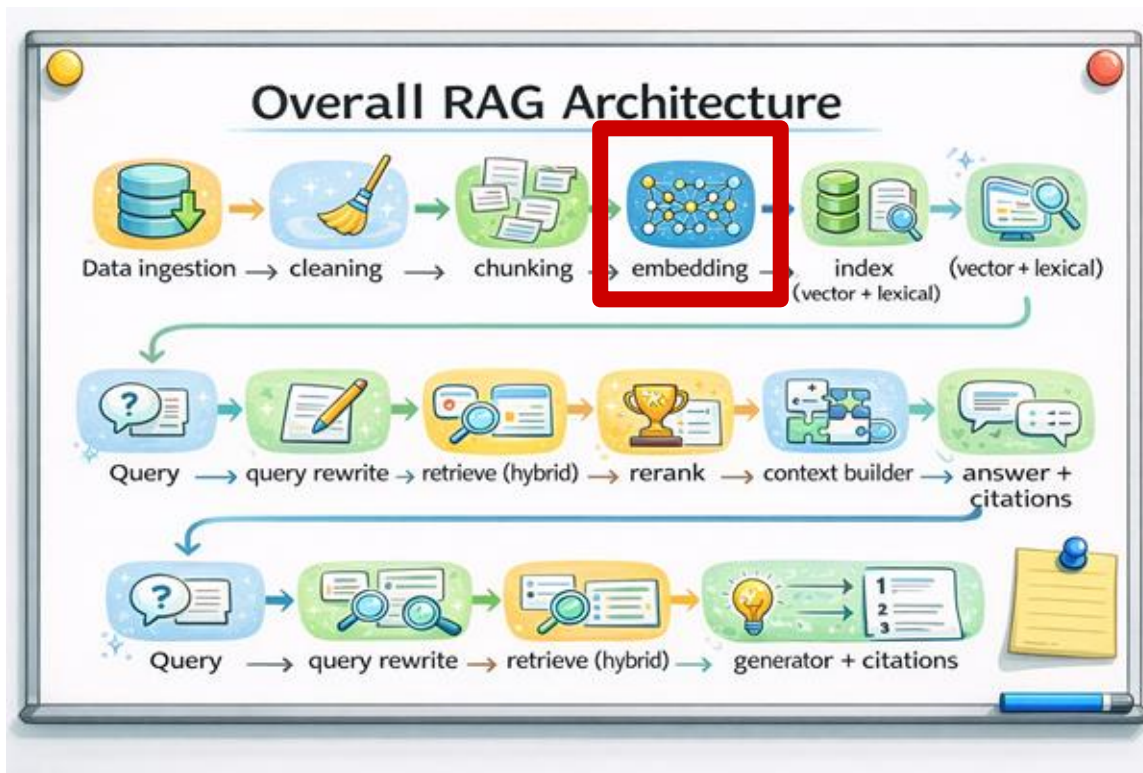
RAG Architecture



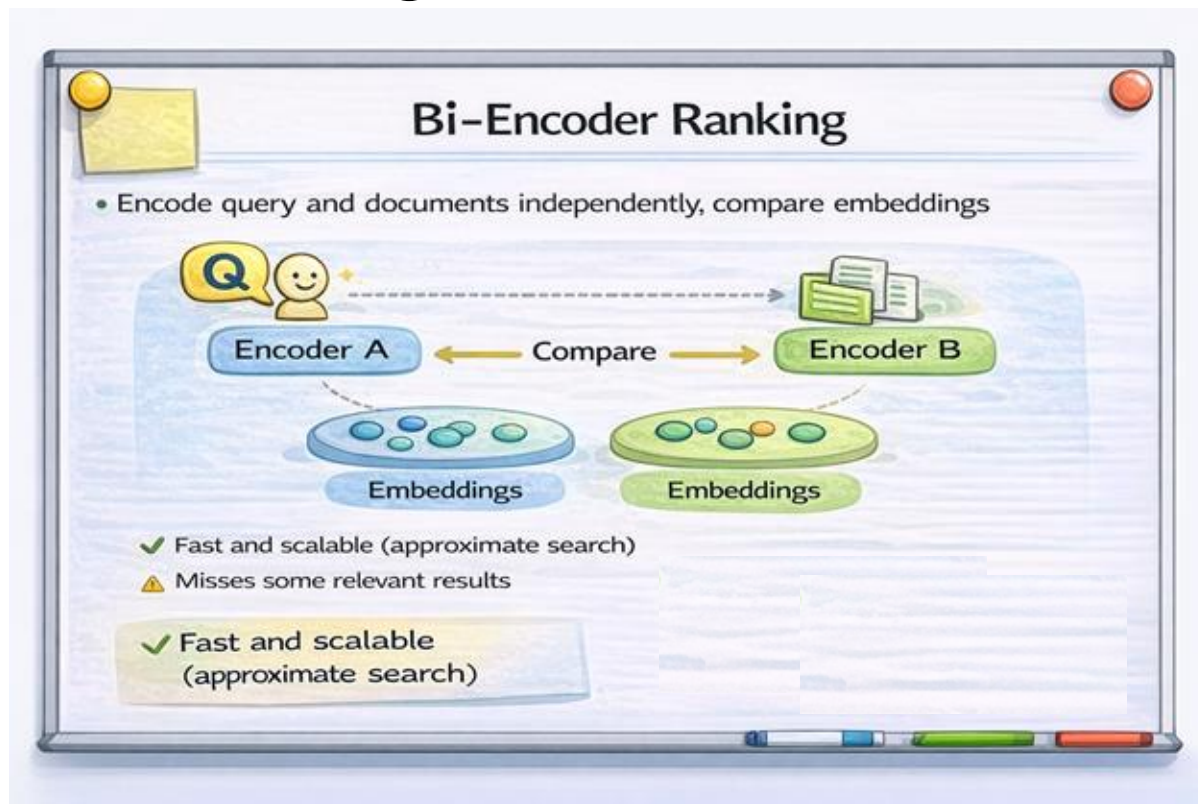
Chunking



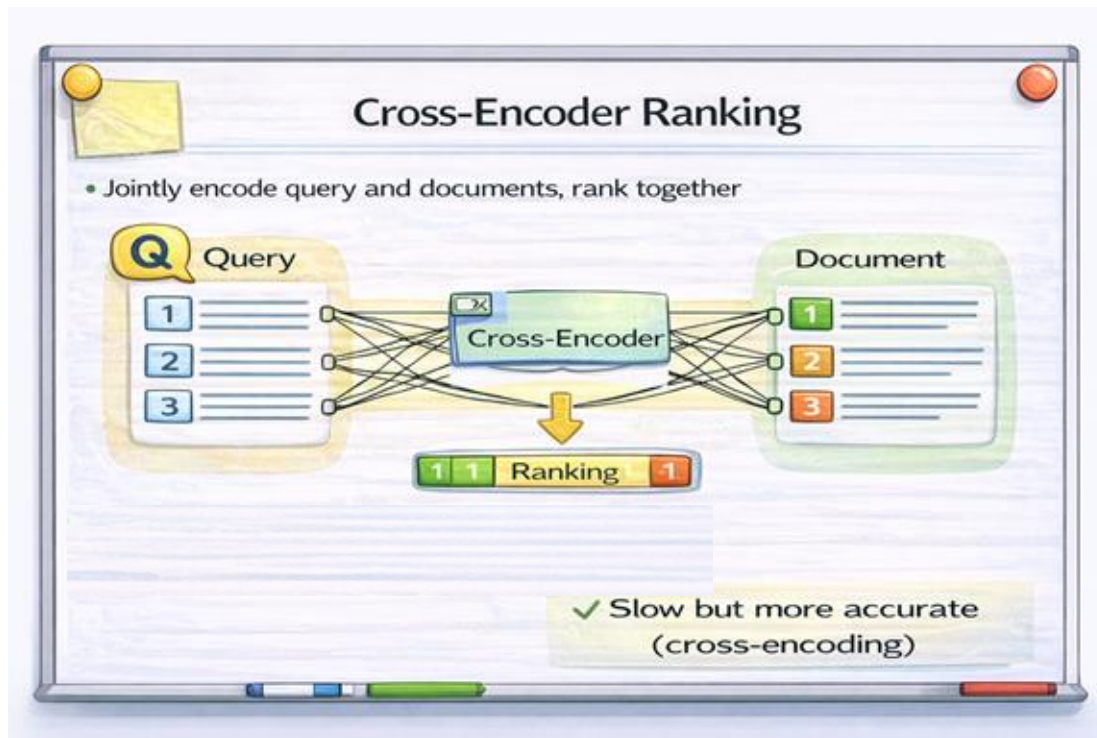
RAG Architecture



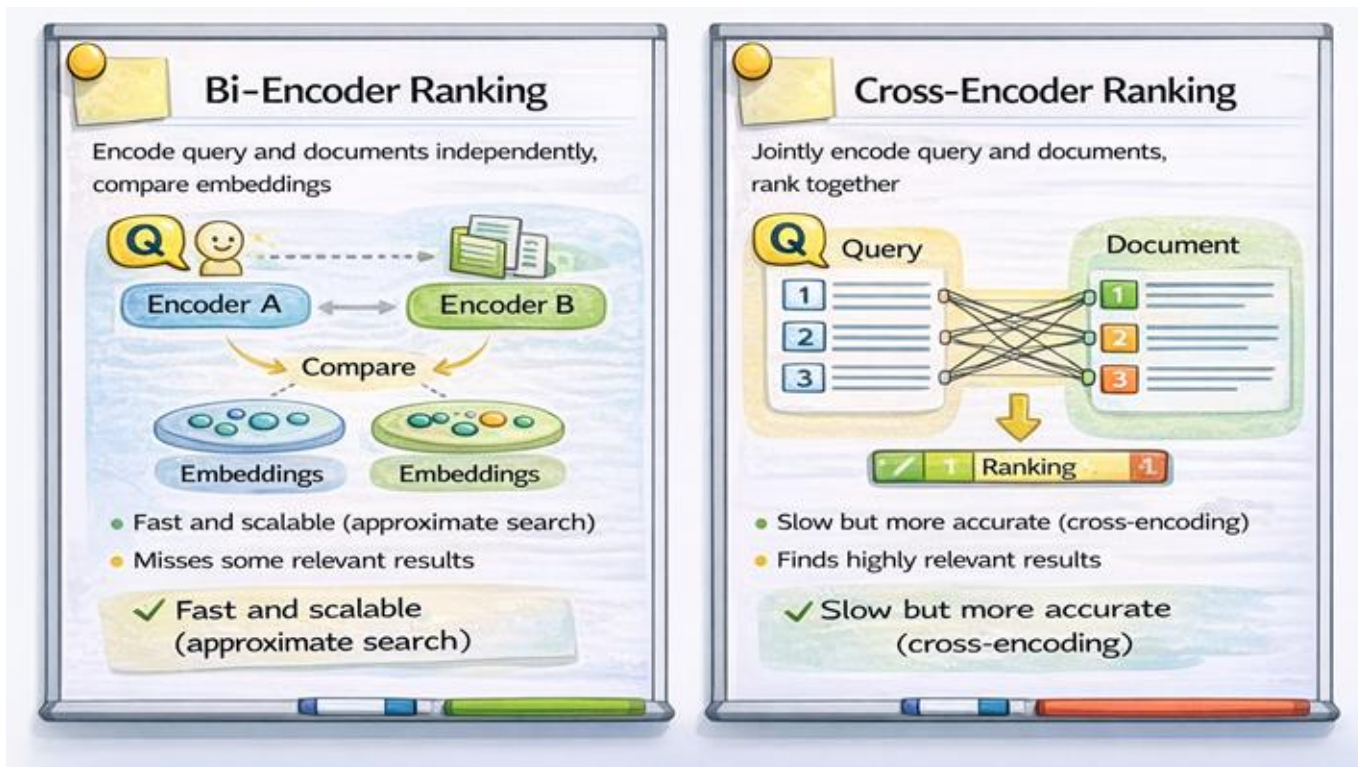
Encoder Ranking



Encoder Ranking



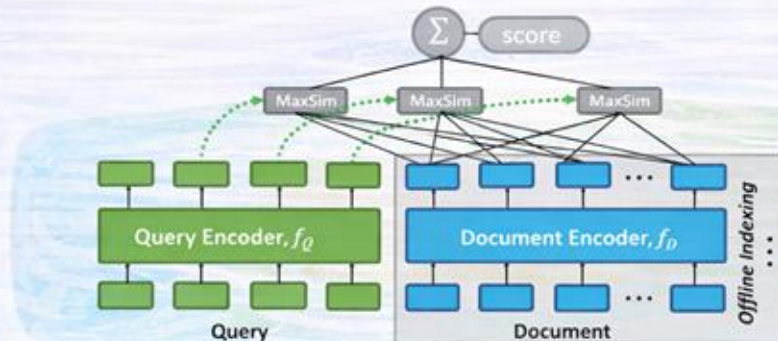
Encoder Ranking



Token-level Dense Retrieval

Token-Level Dense Retrieval

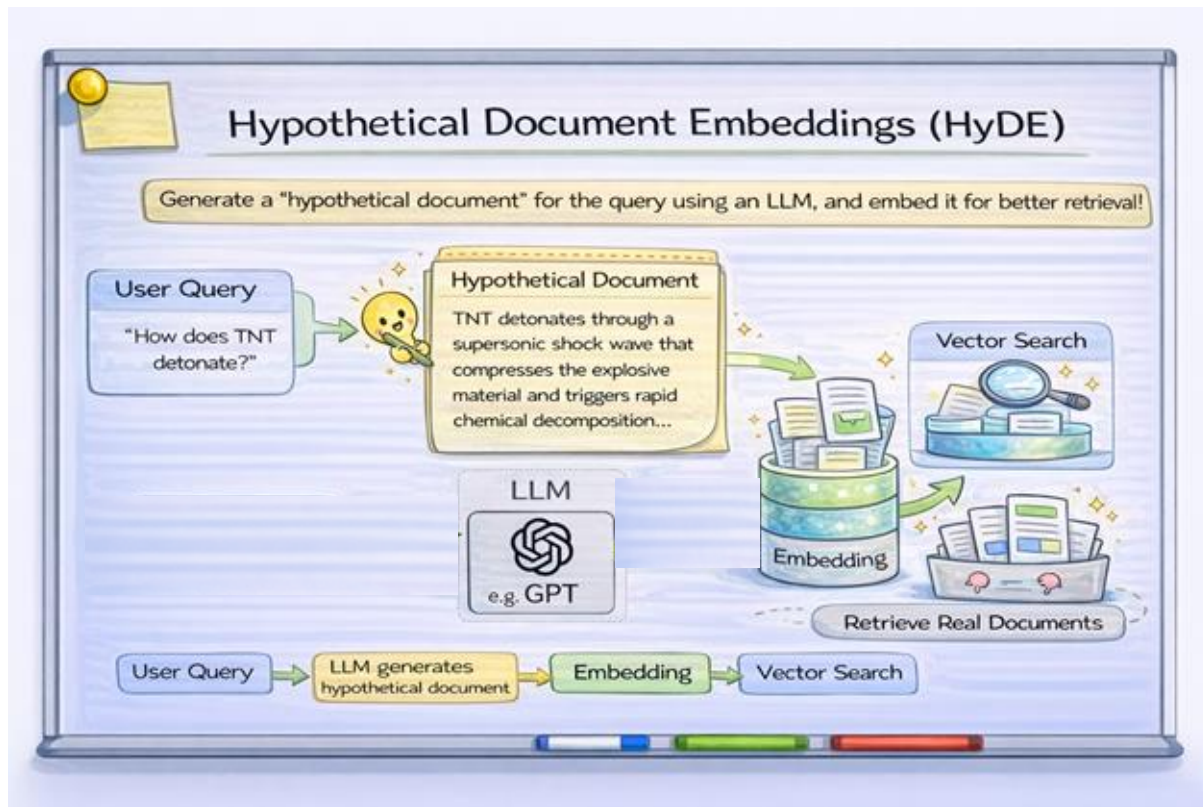
- ColBERT uses contextual representations of all query and document tokens to compute retrieval score.



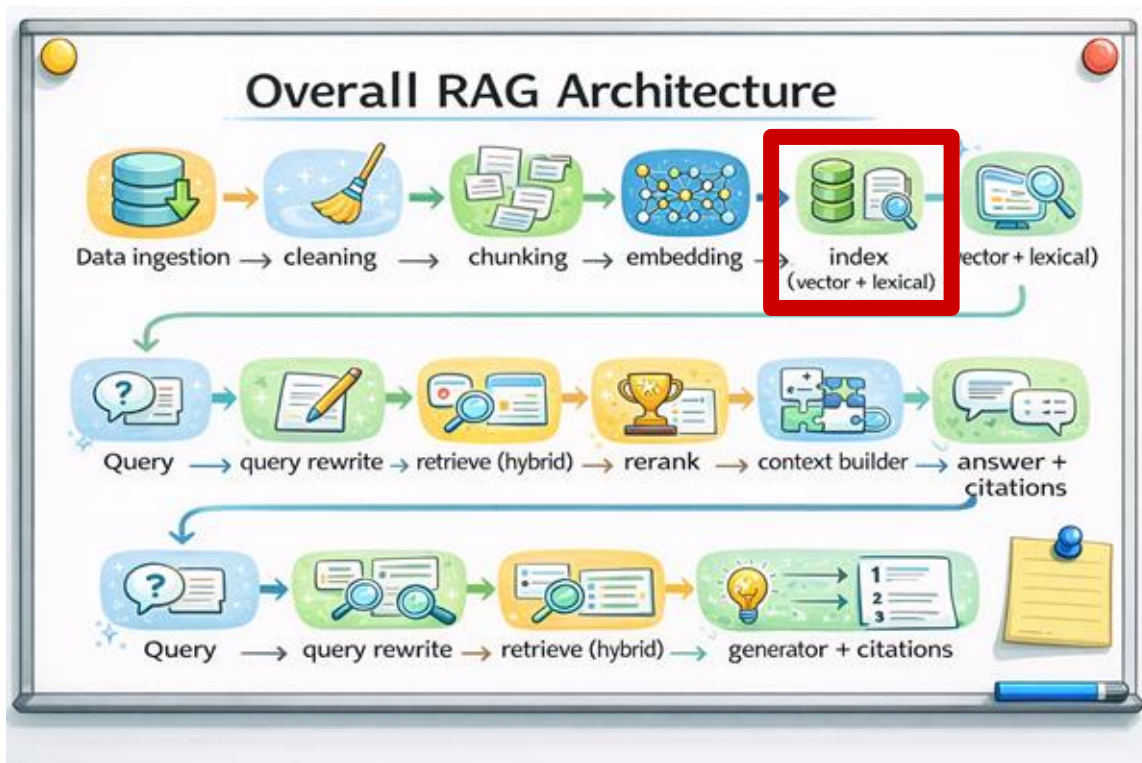
- ✓ Significantly more effective (but more costly) than single-vector retrieval

(Khattab et. 2020)

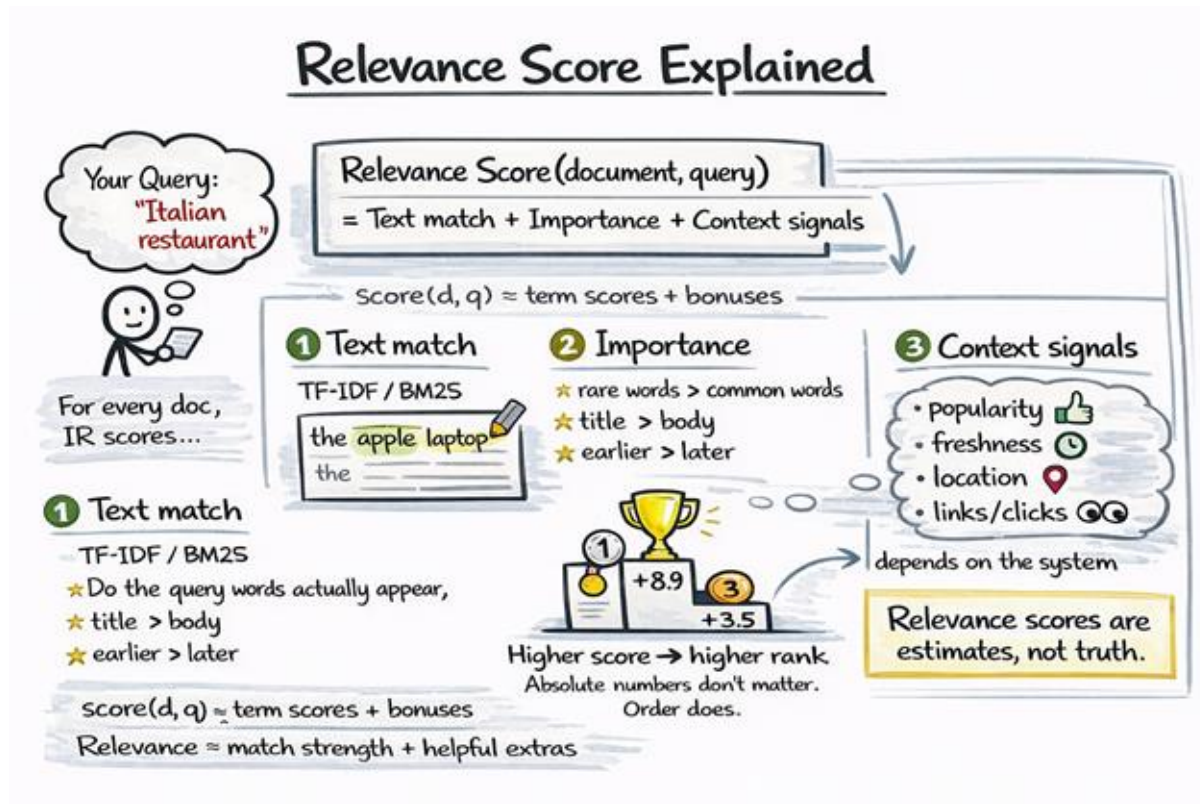
Hypothetical Document Embeddings



RAG Architecture



Relevance score



TD-IDF

TF-IDF : Which words actually matter?

Query: "apple
laptop"



Some words are more helpful than others.

TF = Term Frequency

How often the word appears in **THIS** document

Doc A	Doc B
the the the	the
apple apple	apple
<u>laptop</u>	<u>laptop laptop</u>

IDF = Inverse Document Frequency

How rare the word is across ALL documents

the	the	the	the	the	the	the x
the	the	the	<u>laptop</u>	<u>laptop</u>	the	

▷ If everyone uses it, it's boring.

▷ More appearances → more important locally



$$\text{TF-IDF} = \frac{\text{important here}}{\text{TF} \times \text{IDF}}$$

Specific words push documents up.

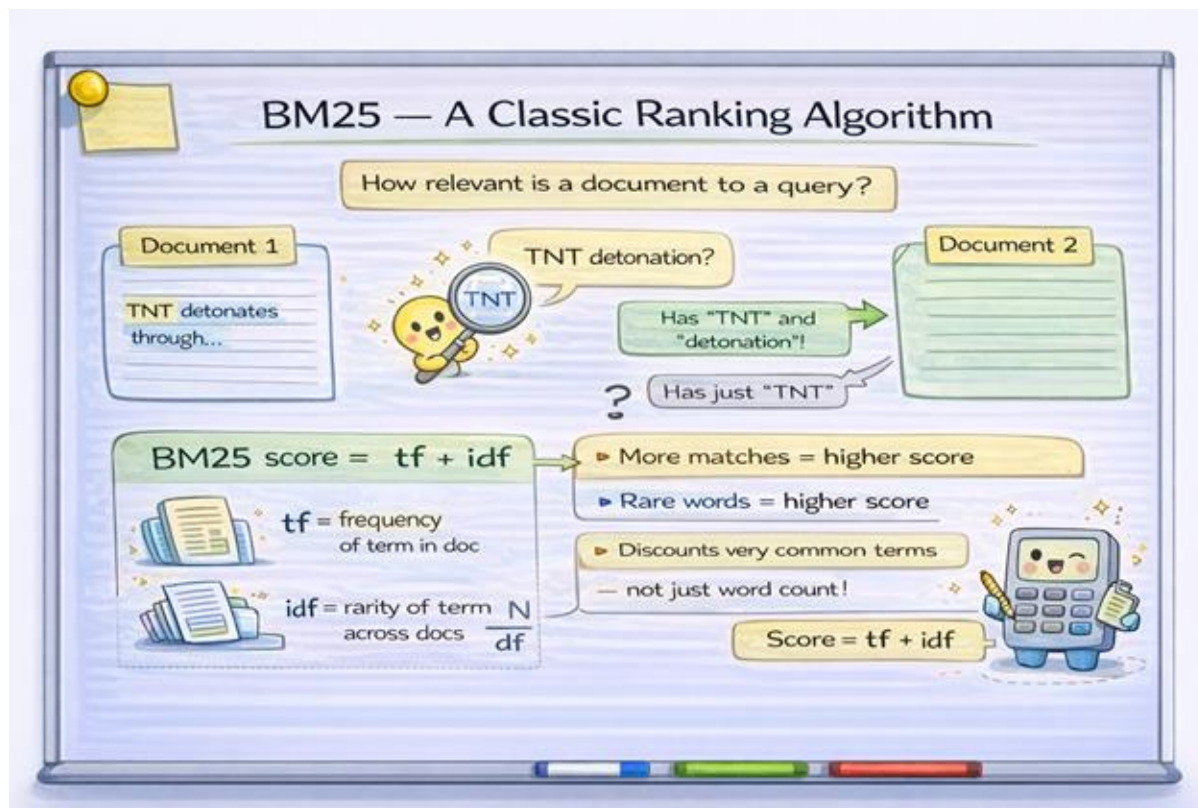
TF-IDF isn't about presence.

It's about discrimination.

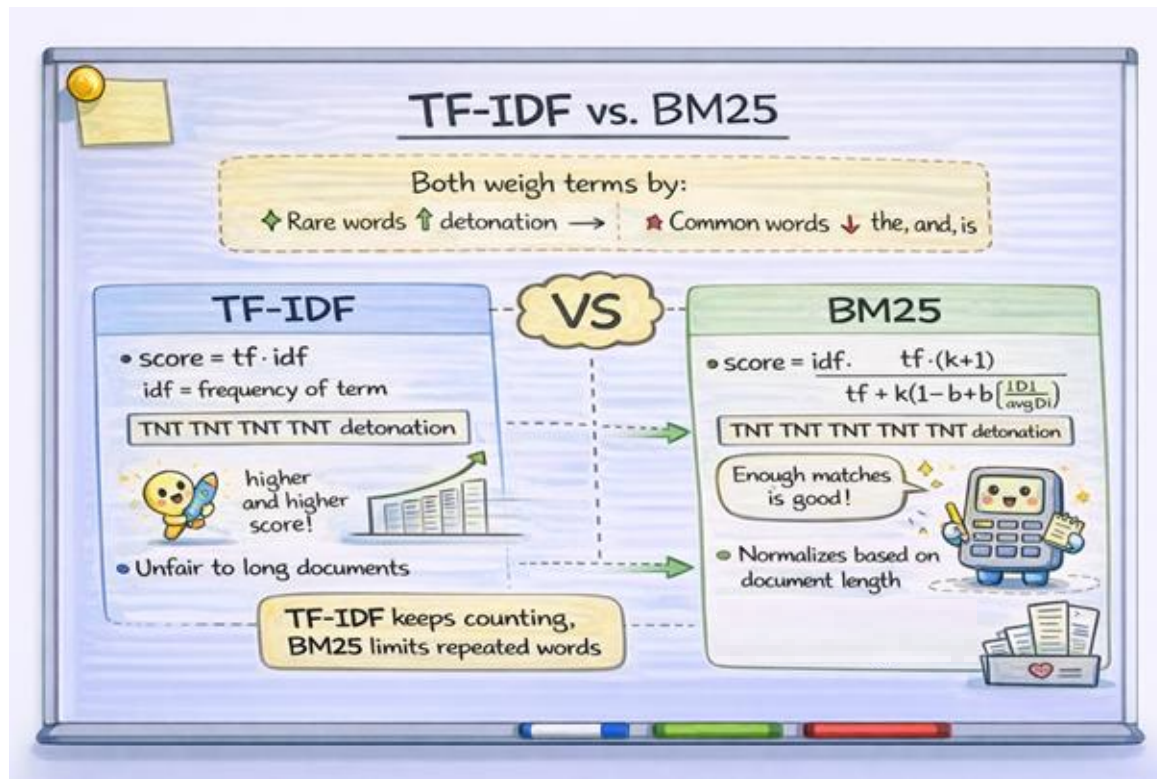
Which words help me tell documents apart?

Stop words are loud but useless. 😊

BM25



TD-IDF vs. BM25



Nearest Neighbor Search

Nearest Neighbor Search (NNS)

"Find the closest vectors."

1 Encode query $\rightarrow$ vector

2 Compute similarity:

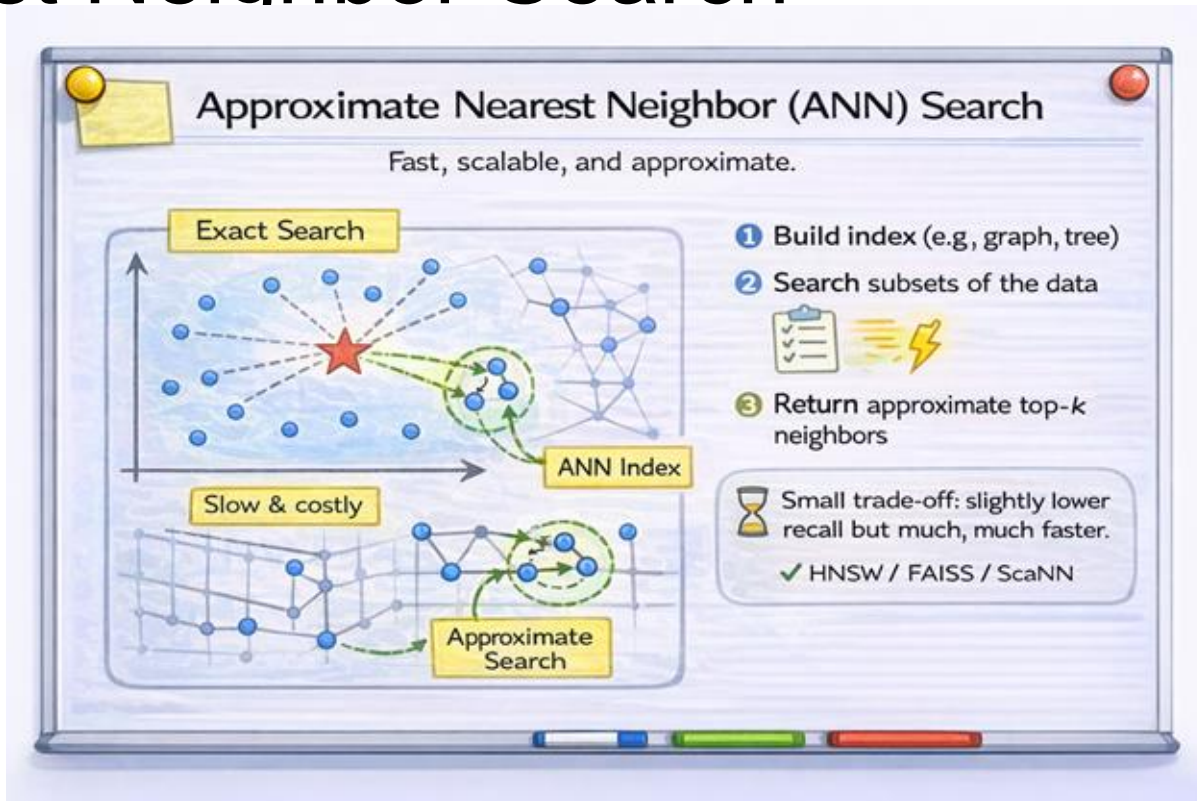
- Cosine similarity
- Dot product
- L2 distance

3 Return Top-k closest vectors

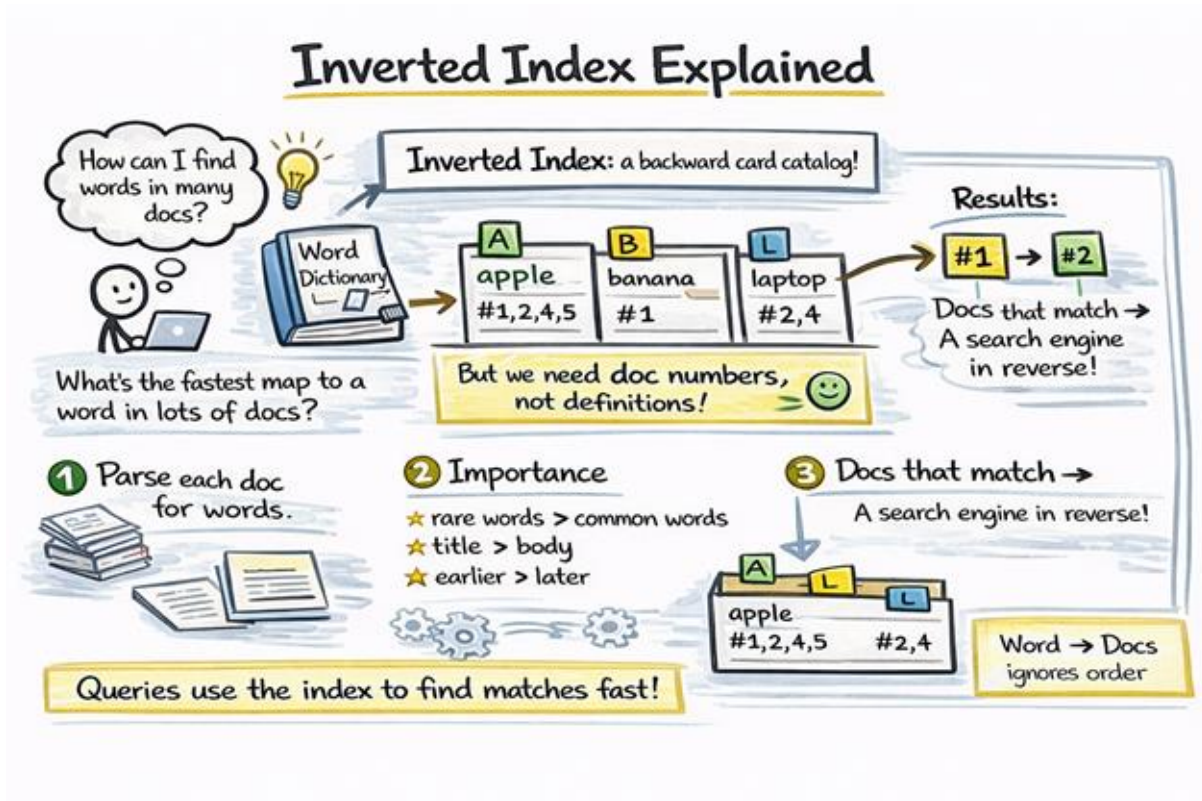
$$NN(q) = \operatorname{argmax}_d \operatorname{sim}(q, d)$$

 Exact search = slow for millions of docs
Approximate search (ANN) = fast & scalable
HNSW / FAISS / ScaNN

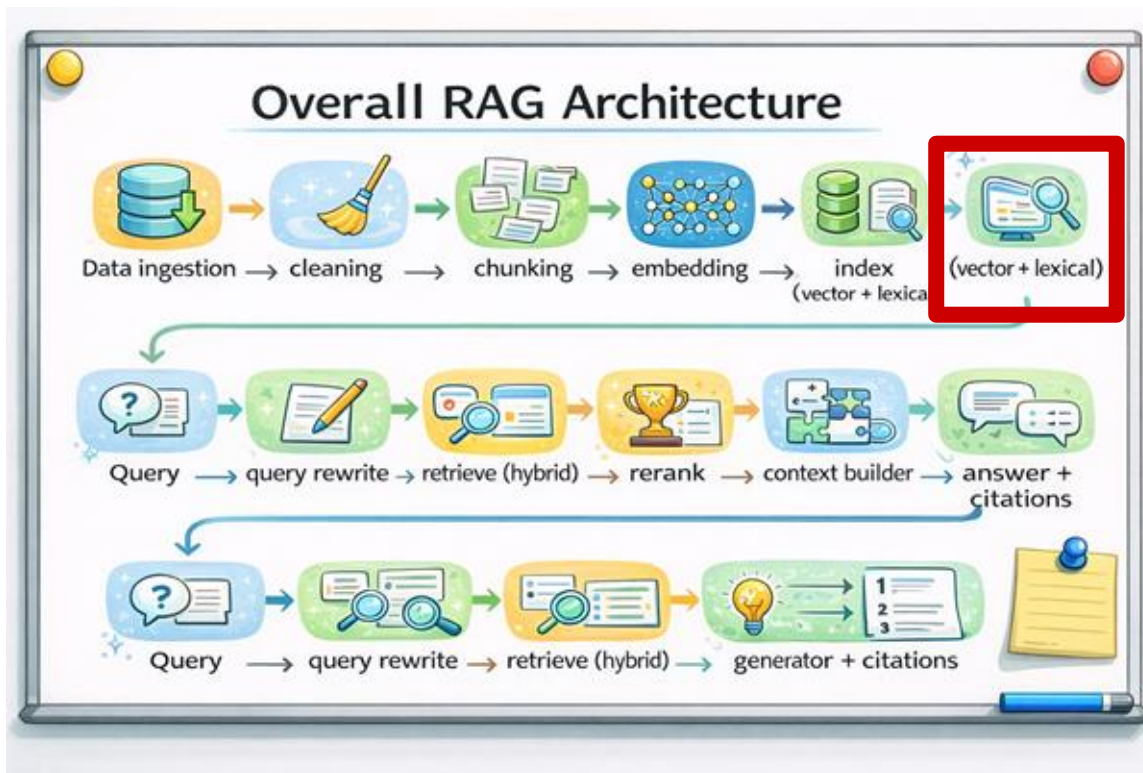
Nearest Neighbor Search



Inverted Indices



RAG Architecture



Dimensions

Many Dimensions

Metrics to balance. Accuracy comes first.



Accuracy

Accuracy metrics: Bullseye!
Key for quality.



Latency

Time to execute a query



Throughput

Queries per second (fastest overall)



FLOPs:

Compute power à la hardware



Disk usage:

For model, index, etc.



FLOPs:

Compute power à la hardware



Disk usage:

For model, index, etc.



Memory usage:

For model, index, etc.



Cost

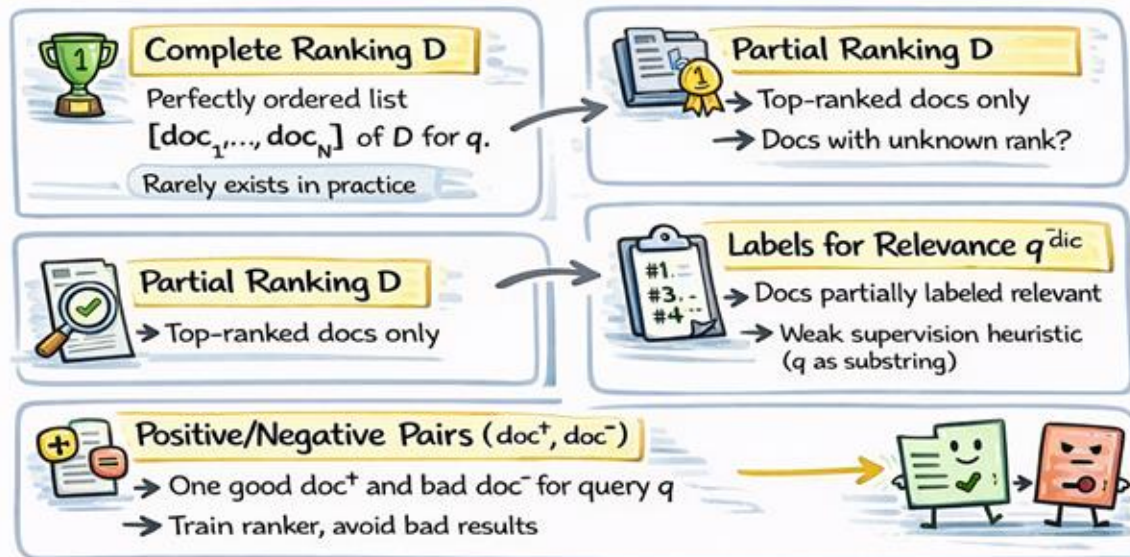
Total
deployment
cost.



Relevance Data types

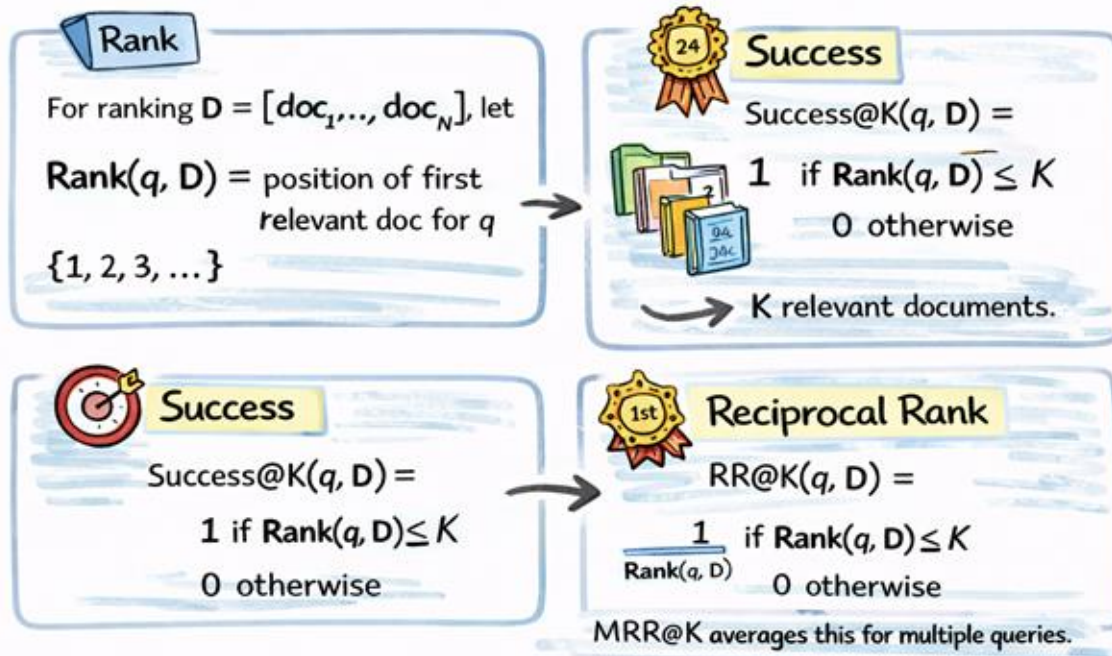
Relevance Data Types

Given a query q and a collection of N documents D :

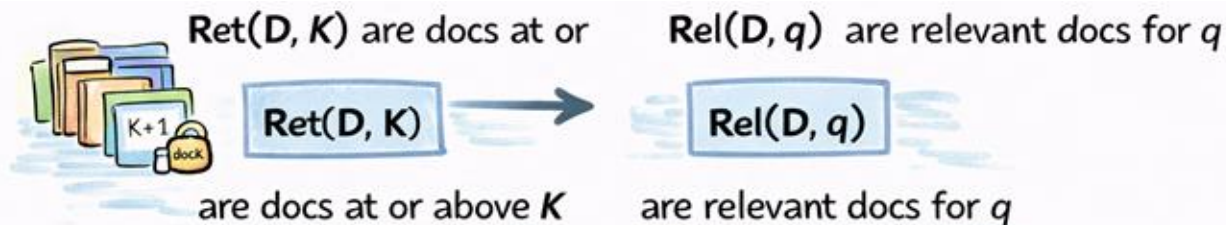


Success and Reciprocal Rank

Success and Reciprocal Rank



Precision and Recall



Precision

$\text{Prec}@K(q, \mathbf{D}) =$

$$\frac{|\text{Ret}(\mathbf{D}, K) \cap \text{Rel}(\mathbf{D}, q)|}{K}$$

Recall

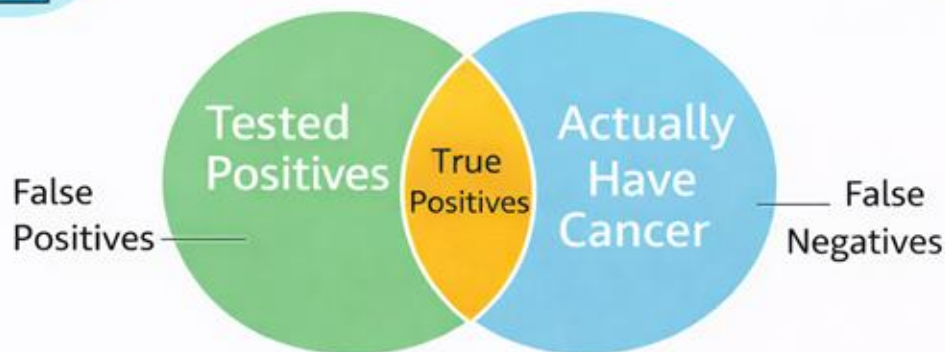
$\text{Rec}@K(q, \mathbf{D}) =$

$$\frac{1}{|\text{Ret}(\mathbf{D}, K)|} \frac{|\text{Ret}(\mathbf{D}, K) \cap \text{Rel}(\mathbf{D}, q)|}{|\text{Rel}(\mathbf{D}, q)|}$$

Precision and Recall: An Example



People who test positive for cancer vs. people who actually have cancer



✓ Precision measures correctness.

$$\text{Precision} = \frac{\# \text{ True Positives}}{\# \text{ Tested Positive}}$$

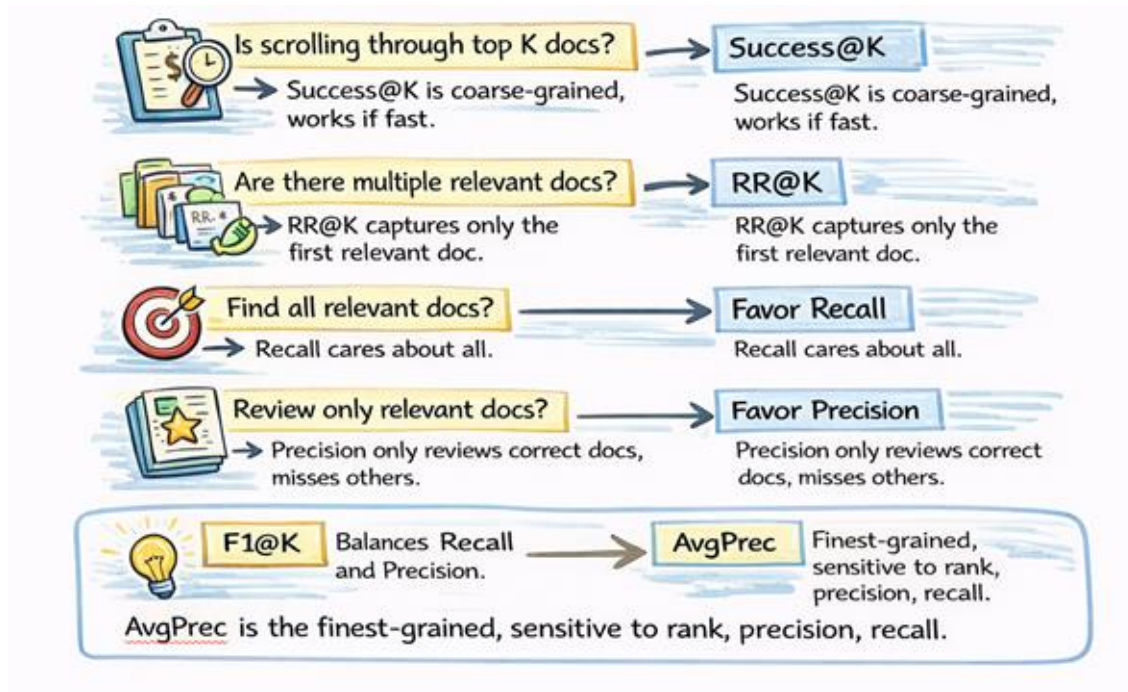
How many testing positive
actually have cancer?

✓ Recall measures coverage

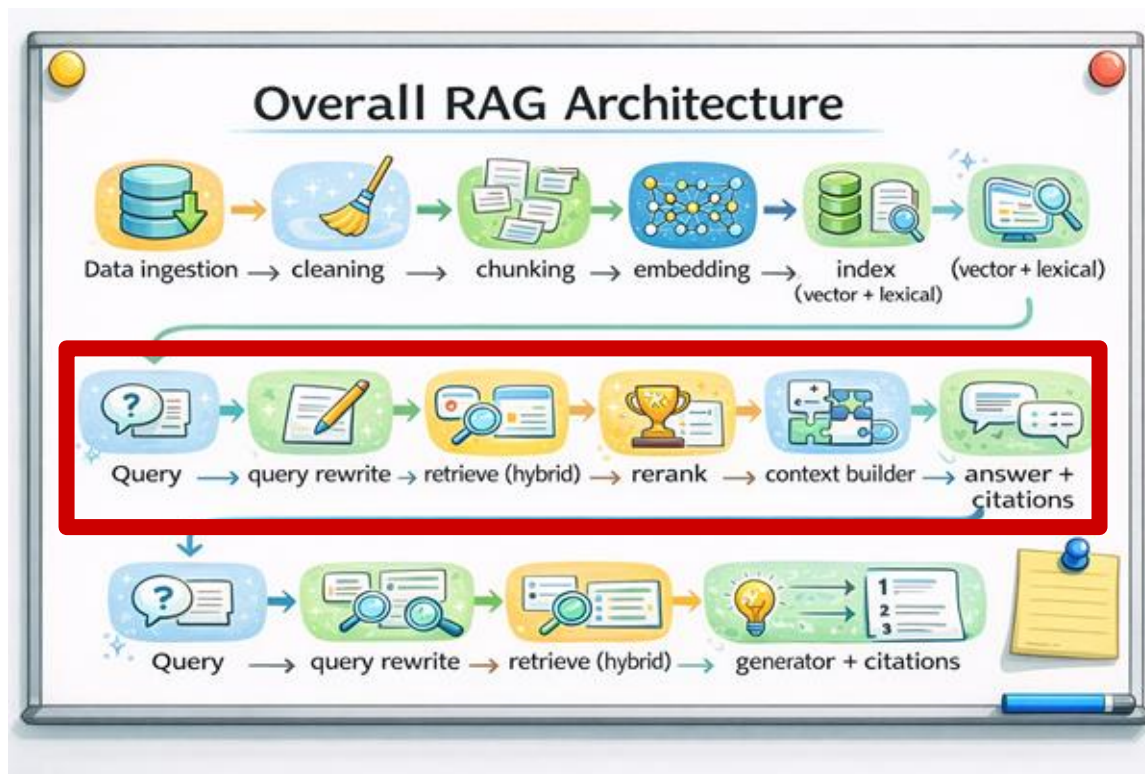
$$\text{Recall} = \frac{\# \text{ True Positives}}{\# \text{ Actually Have Cancer}}$$

How many with cancer were
picked up by the test?

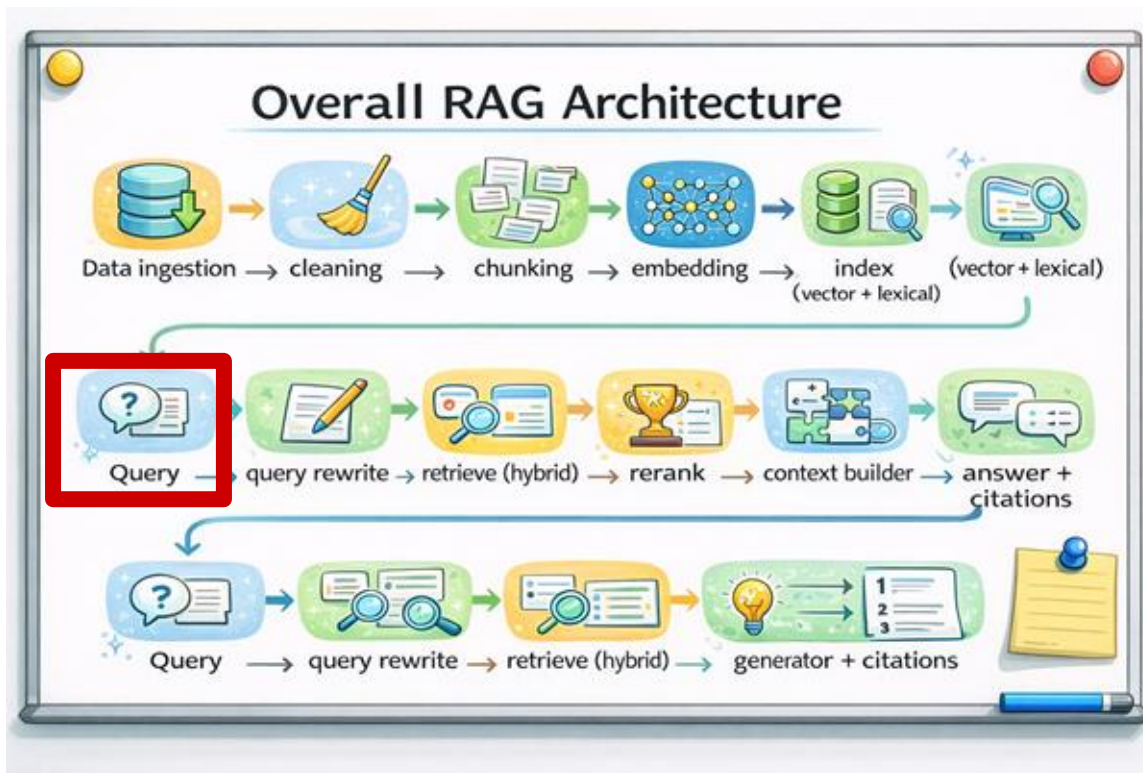
Which metric? There is no single answer!



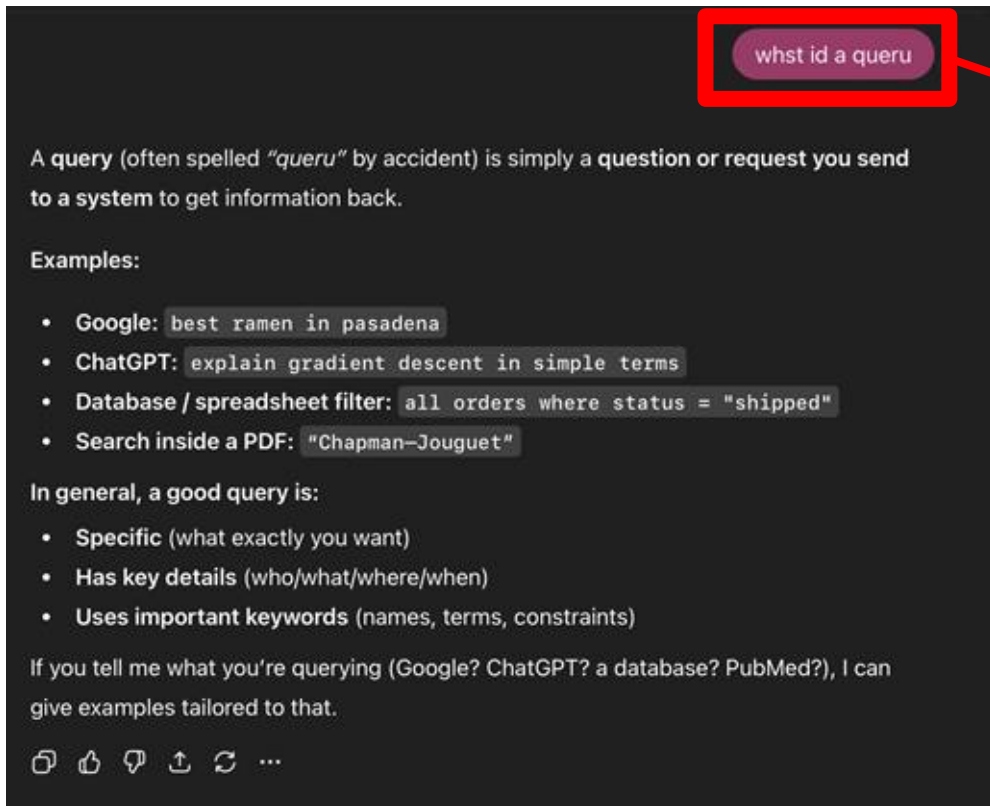
Prompting



Prompting



We all have obviously typed like a maniac....



A screenshot of a chat interface on a dark background. At the top right, a purple speech bubble contains the text "whst id a queru", which is highlighted with a red rectangular box. A red line connects this box to a larger purple speech bubble on the right side of the screen, also containing the text "whst id a queru" and highlighted with a red rectangular box. The main chat area contains the following text:

A **query** (often spelled "*queru*" by accident) is simply a question or request you send to a system to get information back.

Examples:

- Google: `best ramen in pasadena`
- ChatGPT: `explain gradient descent in simple terms`
- Database / spreadsheet filter: `all orders where status = "shipped"`
- Search inside a PDF: `"Chapman-Jouquet"`

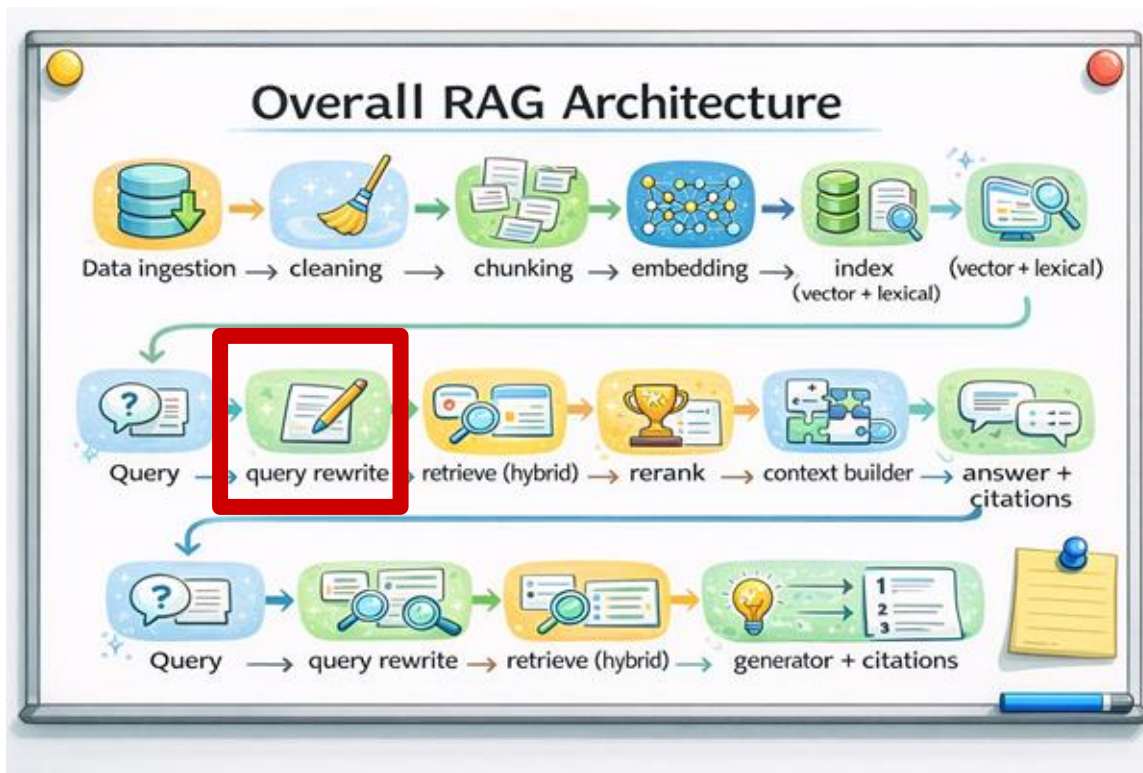
In general, a good query is:

- **Specific** (what exactly you want)
- **Has key details** (who/what/where/when)
- **Uses important keywords** (names, terms, constraints)

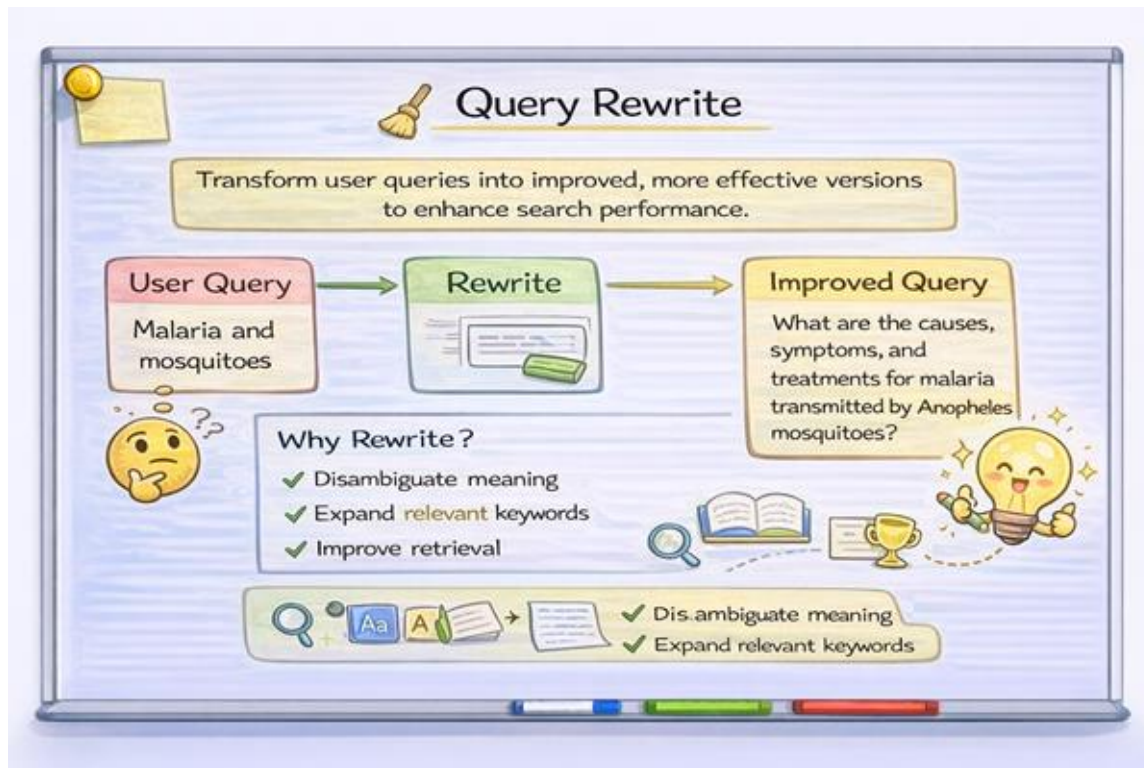
If you tell me what you're querying (Google? ChatGPT? a database? PubMed?), I can give examples tailored to that.

At the bottom of the chat area, there is a row of small icons: a copy icon, a thumbs up icon, a thumbs down icon, an up arrow icon, a refresh icon, and an ellipsis icon.

RAG Architecture



Rewrite user query



Beyond term matching



Query and document expansion



Phrase search



Term dependence



Different document fields (e.g., title, body)



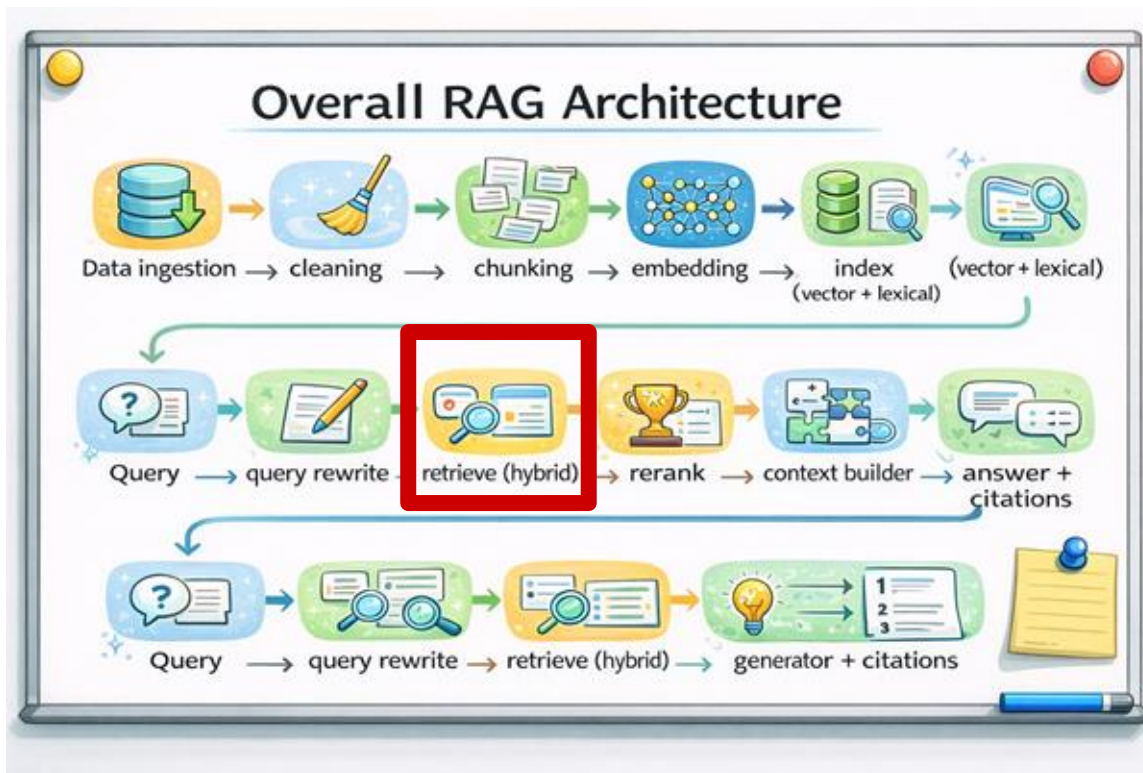
Link analysis (e.g., PageRank)



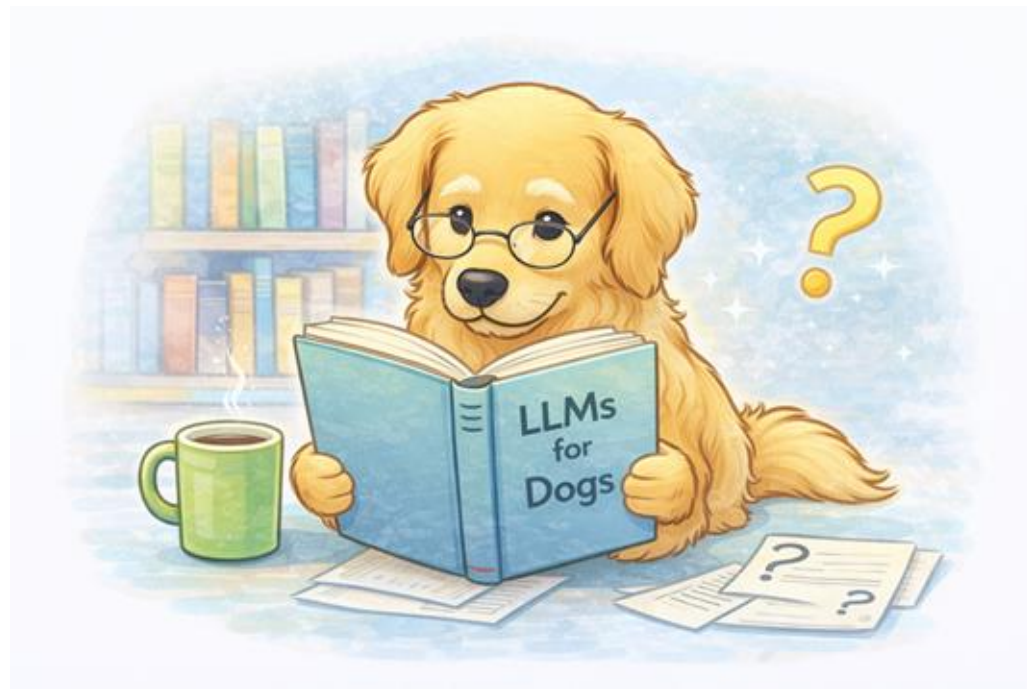
Learning to rank



RAG Architecture

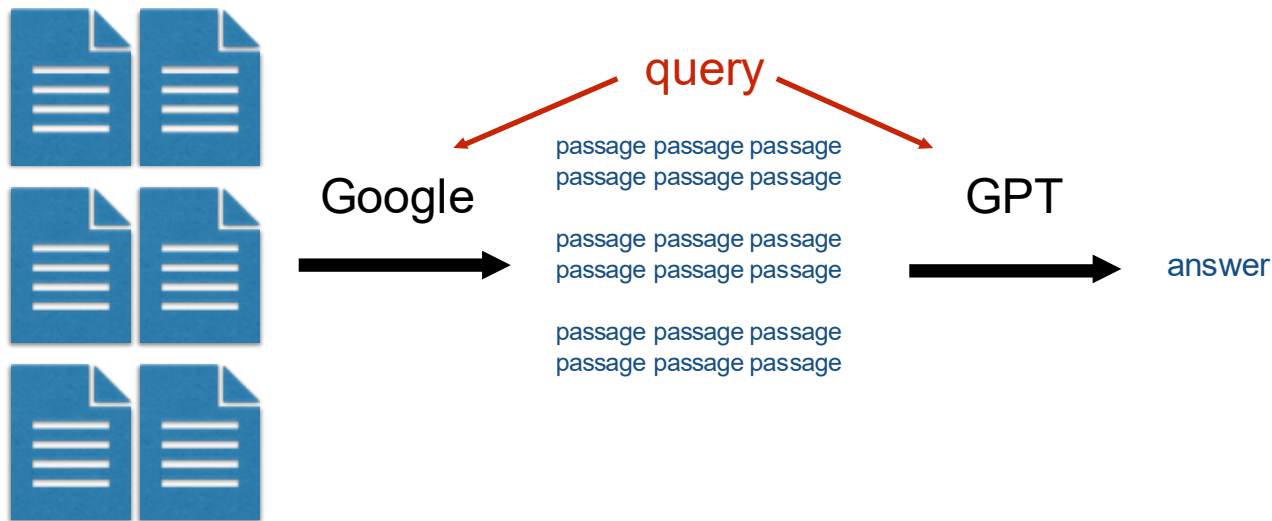


Retriever-Reader Models



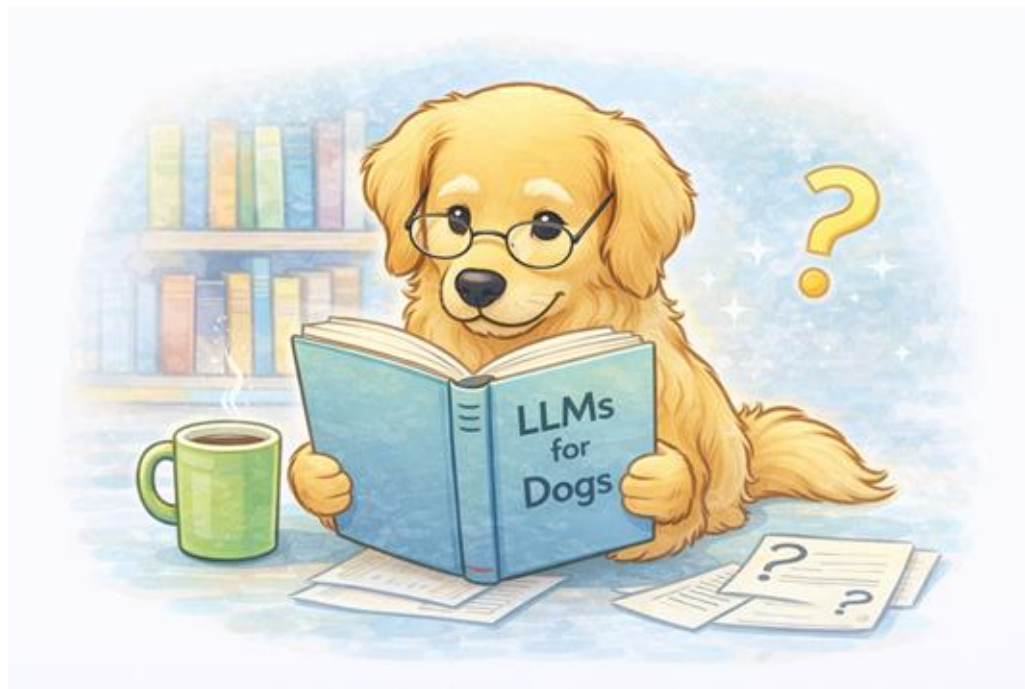
Simple: Just Chain Retrieval+Reading

- Use an out-of-the-box retriever and out-of-the-box reader



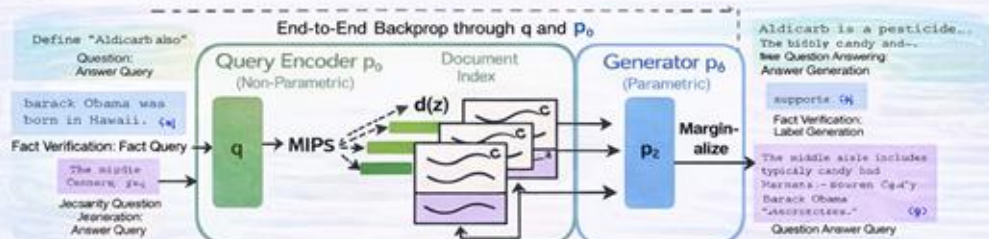
- Passages are concatenated to the context

The “Golden” Retriever-Reader Model



Retriever + Generator End-to-end Training

- Train the retriever and reader to improve accuracy
 - **Reader:** Maximize generation likelihood given single retrieved document
- **Retriever:** Maximize overall likelihood by optimizing mixture weights over documents



✓ Combines retrieval accuracy & generation capacity

⚠ High computational cost due to end-to-end training

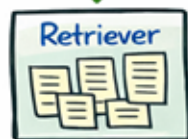
(Lewis et al. 2020)

End-to-end Training Equations

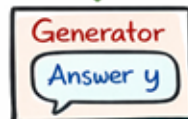
End-to-End Training in RAG

- Generation combines **retrieval** + **language modeling**

Query x



Retriever
selects $dc\ z$



Generator
produces
answer y

$$P_{\text{RAG}}(y|x) = \sum_z p_{\eta}(z|x) \cdot p_{\theta}(y|x, z)$$

- ✓ **Retriever** picks relevant documents
- ✓ **Generator** writes the answer using document

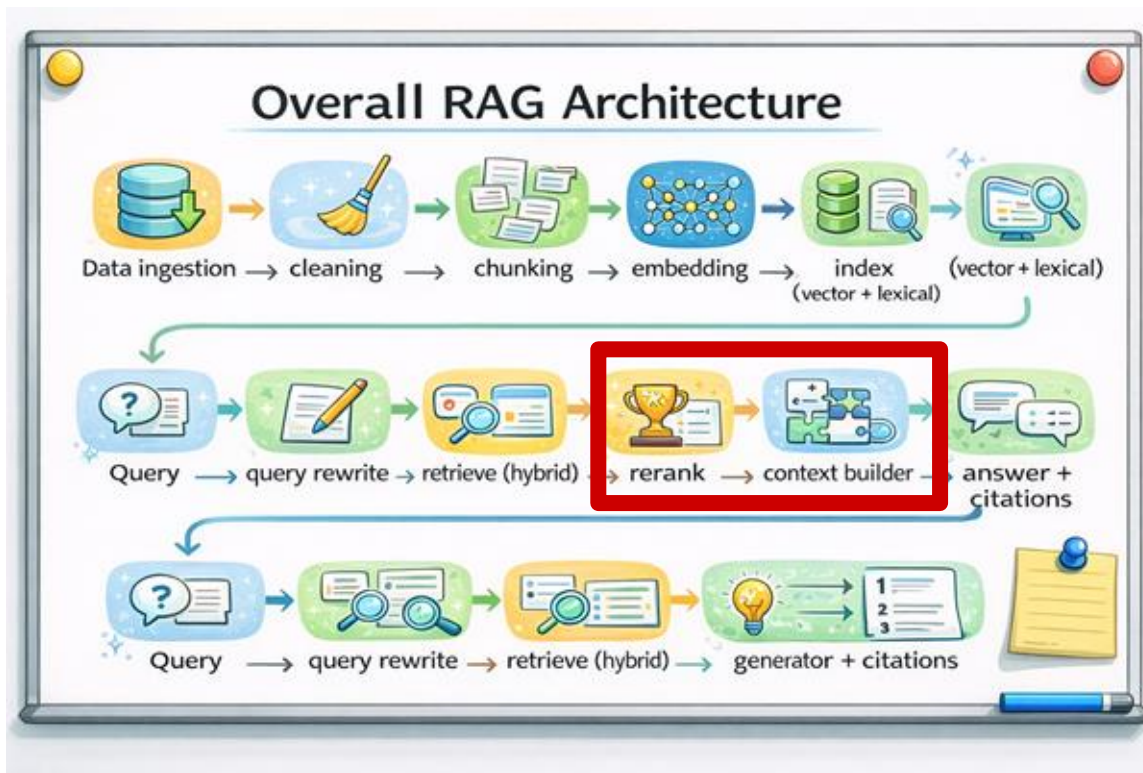
Retriever probability $\times$ Generator probability

- Model marginalizes over possible documents
- Considers multiple retrieved documents

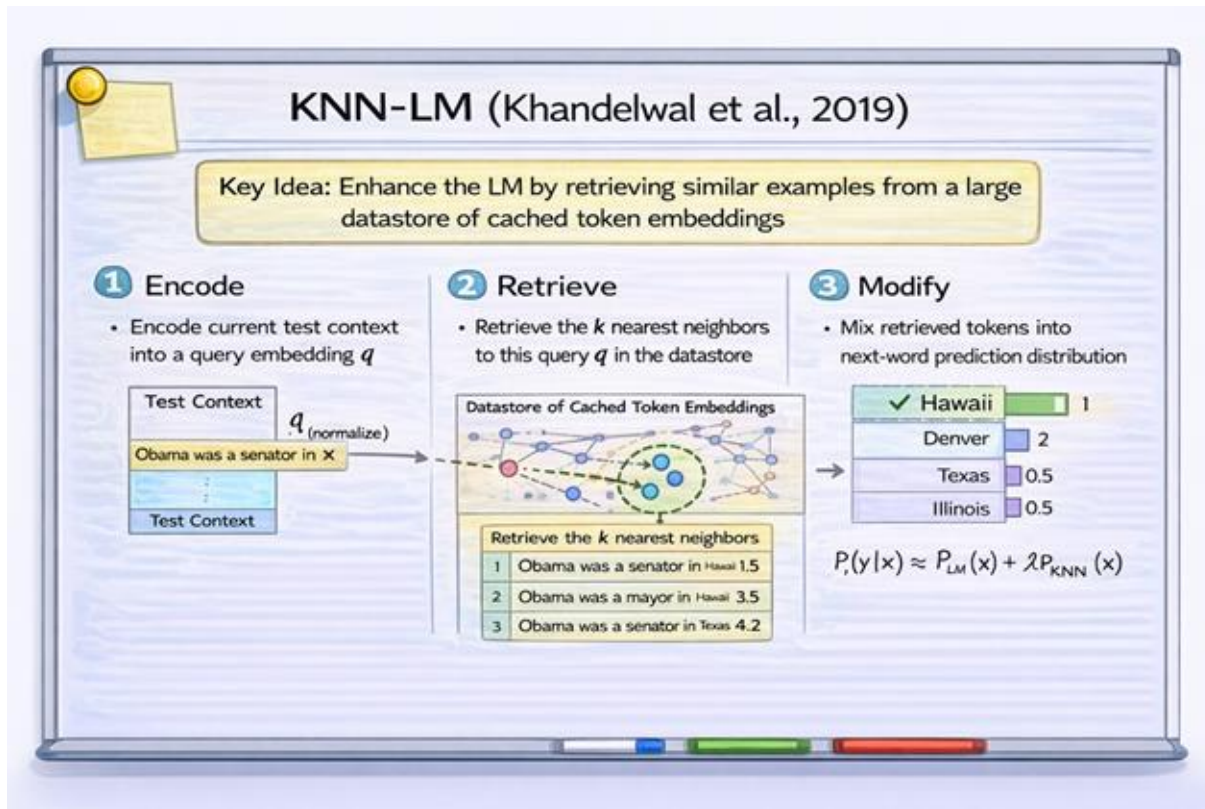
When do we Retrieve?

- **Once, at the beginning of generation**
 - Default method used by most systems (Lewis et al. 2020)
- **Several times during generation, as necessary**
 - Generate a search token (Schick et al. 2023)
 - Search when the model is uncertain (Jiang et al. 2023)
- **Every token**
 - Find similar final embeddings (Khandelwal et al. 2019)
 - Approximate attention with nearest neighbors (Bertsch et al. 2023)

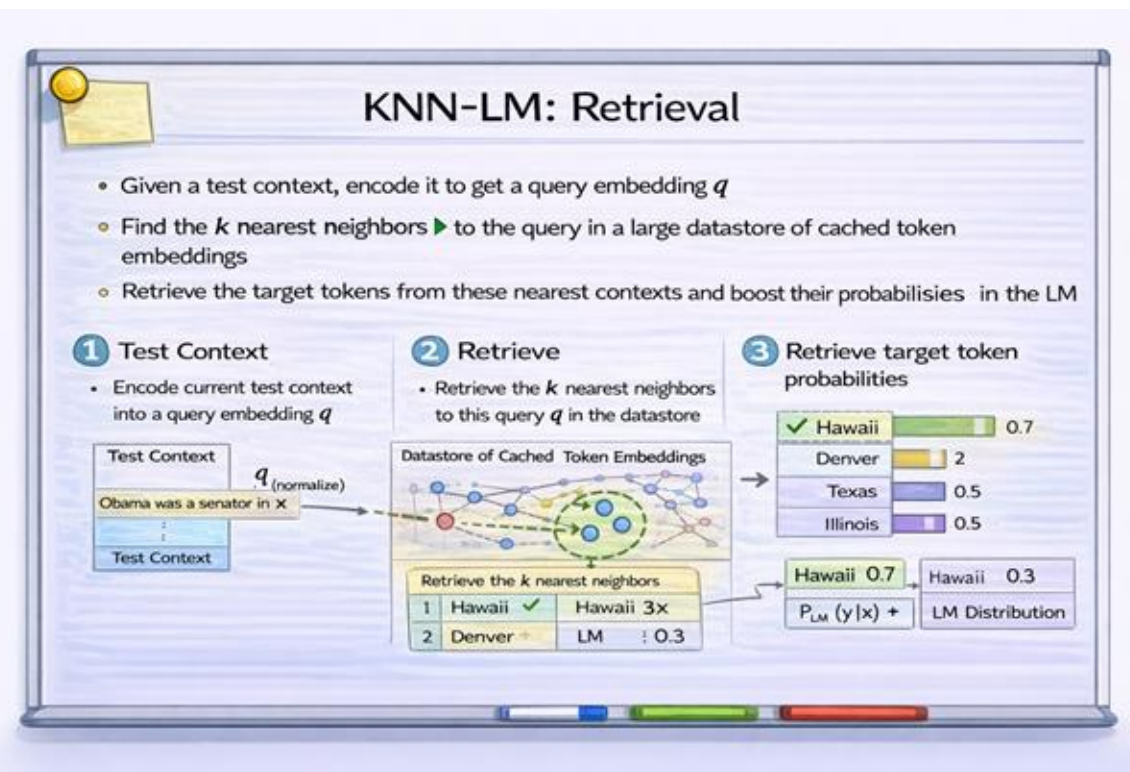
RAG Architecture



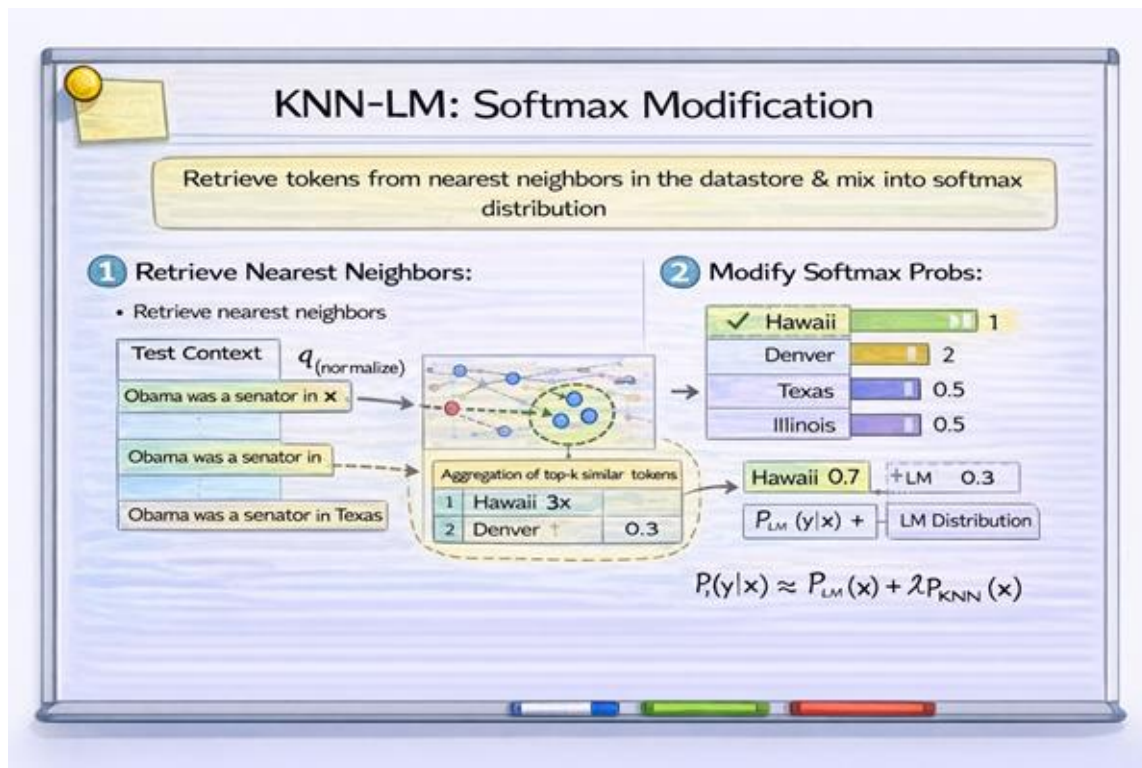
KNN-LM: An overview



KNN-LM: Retrieval



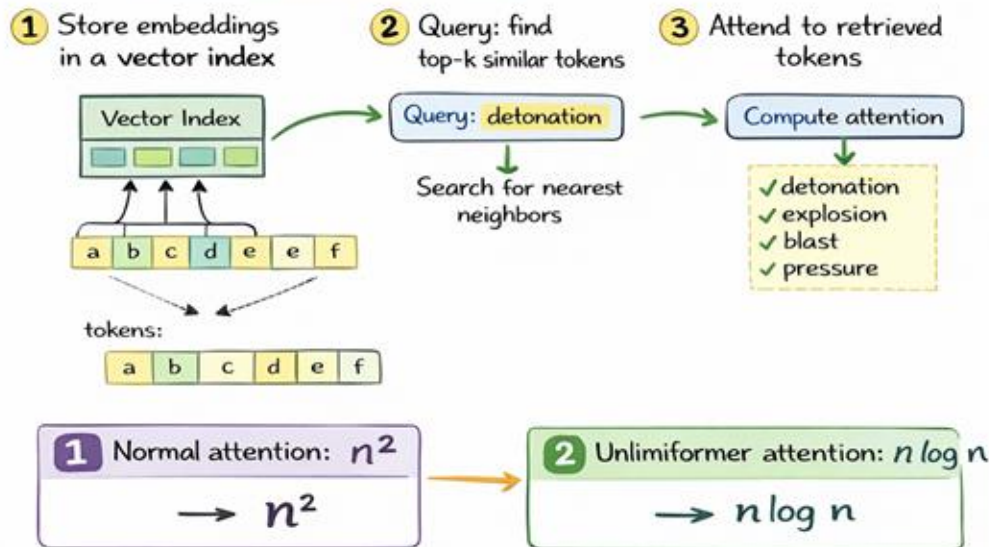
KNN-LM: Softmax Modification



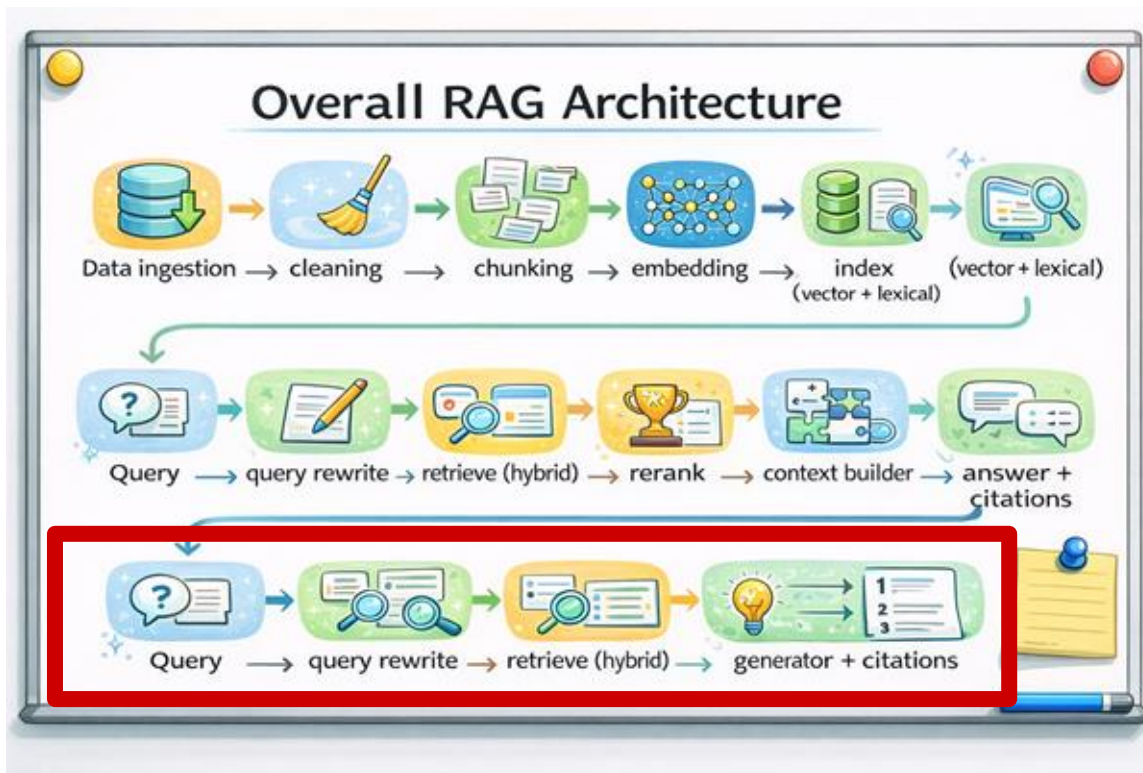
Token-level Approximate Attention

Token-level Approximate Attention

How can we speed up attention in long documents?



Finished RAG Pipeline



Summary

